

cargo-cicd: A Level-5 Process-Data Engine for Local-First CI/CD Orchestration in Rust Workspaces

PhD Thesis

Department of Computer Science

School of Software Engineering

Version 26.6.2 — June 2026

Chapter 1: Introduction

Abstract

Continuous integration and continuous delivery (CI/CD) pipelines have become indispensable infrastructure in modern software development, yet the feedback loop they impose is almost universally remote-first: a developer commits, pushes, and waits for a cloud pipeline to adjudicate quality. For Rust workspaces in particular, this round-trip is expensive — compilation times are long, toolchain mismatches surface late, and bloated `target/` directories silently inflate both local and remote build times. This thesis presents `cargo-cicd`, a local-first CI/CD orchestration engine for Rust workspaces that shifts quality enforcement to the developer's machine, before the push. The system is architected as a Level 5 process-data engine: all runtime dimensions — workspace health, toolchain state, target directory pressure, changed-file sets, git phase, and autonomic policy verdicts — are unified into a single aggregate root (`EngineState`) populated by a pipeline of pure translation adapters. A noun-verb command grammar, built atop the `clap-noun-verb` library, exposes this engine to the developer as an ergonomic Cargo subcommand. Workspace state is serialised to a local carrier file (`cicd.toml`) and, under the optional `wasm4pm` feature flag, process evidence is emitted as XES (XML Event Stream) event logs and adjudicated by an external oracle. We define three core research contributions: (1) a formal adapter-pipeline architecture for translating heterogeneous external sources into a unified engine state; (2) an autonomic policy layer that issues non-destructive workspace recommendations in suggest mode; and (3) an evidence-gate pattern that connects local process events to external adjudication, enabling verifiable release closure. Evaluation across representative Rust workspaces demonstrates that `cargo-cicd` detects the dominant classes of push-blocking defects — dirty trees, stale trybuild fixtures, and target bloat — with zero false negatives in the invariant test suite. The system is implemented in Rust (MSRV 1.86), ships at version 26.6.2, and is available under the MIT/Apache-2.0 dual license.

1. Introduction

1.1 Motivation

The modern software release pipeline is, in its dominant form, a remote-first construct. Developers author code locally but delegate quality enforcement to cloud-hosted runners — GitHub Actions, GitLab CI, CircleCI — that execute builds, tests, and static analysis only after a commit is pushed to a remote branch [1]. This architecture places the feedback loop at the wrong end of the development cycle. For compiled, statically typed languages such as Rust, the cost of a remote-gated defect is compounded by compilation latency: even on well-resourced runners, full workspace builds can take minutes, and the developer must context-switch back to a local editor after an arbitrary delay [2].

Rust's type system and borrow checker eliminate whole categories of runtime defect [3], but they do not eliminate the class of defect that `cargo-cicd` targets: workspace hygiene defects. These include dirty working trees that would fail a remote `git status` check, `target/` directories that have grown to tens of gigabytes and saturate CI cache budgets, `rust-toolchain.toml` mismatches that pass locally on a pinned channel but fail on a runner's stable toolchain, and `trybuild` compile-fail fixtures that bit-rot between refactors [4]. None of these defects are caught by `cargo test` or `cargo clippy` alone. They are caught — expensively — by the remote pipeline, or not caught at all until a publish attempt fails.

The hypothesis motivating this work is that a local-first quality enforcement layer, integrated into the developer's pre-push workflow and implemented as a native Cargo subcommand, can eliminate the dominant classes of push-blocking defect at negligible marginal cost. The developer invocation is a single command:

```
cargo cicd status
```

This command surfaces a structured verdict — `pass`, `warn`, or `refuse` — drawn from a fully populated `EngineState` in under a second for typical workspaces. The intent is not to replace remote CI but to make the common case of "push-ready?" answerable without a network round-trip.

1.2 Problem Statement

Three concrete problems motivate the design of `cargo-cicd` :

Problem 1: Semantic gap between local and remote workspace state. A Rust developer working locally has access to the full workspace state — git porcelain output, `target/` directory metadata, `rust-toolchain.toml` channel, Cargo manifest validity — but no unified tool aggregates these dimensions into an actionable pre-push verdict. Existing tools (`cargo clippy` , `cargo test` , `git status`) address individual dimensions in isolation [5]. The developer must mentally compose their outputs, which is error-prone and adds cognitive overhead at exactly the moment when she is focused on feature delivery.

Problem 2: Absence of process evidence for release verification. Mature process engineering disciplines, including ISO 9001 and DO-178C avionics software standards, require auditable evidence of process execution [6, 7]. Rust workspaces lack a standard mechanism for emitting structured, machine-verifiable evidence of CI/CD process steps. Without such evidence, release claims ("all tests passed before this tag") rest on ephemeral CI logs that may be unavailable, incomplete, or unverifiable by downstream consumers.

Problem 3: Reactive rather than proactive quality enforcement. Existing CI/CD integrations for Rust workspaces are reactive: they execute in response to push or pull-request events and report failure after the fact [8]. A proactive, autonomic layer that monitors workspace state and issues recommendations — without destructive side effects — would shift quality enforcement earlier in the development loop, reducing the mean time to feedback.

1.3 Research Questions

This thesis addresses four research questions:

RQ1. Can a unified aggregate-root state model (`EngineState`) faithfully represent all runtime dimensions of a Rust workspace's CI/CD readiness, sourced from heterogeneous external adapters, with sufficient fidelity to drive a correct `pass / warn / refuse` verdict?

RQ2. Does a noun-verb command grammar, exposed as a Cargo subcommand, provide a sufficiently low-friction developer interface that adoption does not require workflow modification beyond inserting a single pre-push command?

RQ3. Can XES-format process-evidence emission, combined with an external adjudication oracle, provide machine-verifiable release closure that is independent of the `cargo-cicd` tool itself, eliminating the risk of self-certification?

RQ4. Does an autonomic policy layer operating in suggest mode — issuing recommendations without taking action — provide actionable workspace intelligence without introducing the risk of destructive automation in the developer's local environment?

1.4 Contributions

This thesis makes the following original contributions:

1. **The adapter-pipeline architecture.** We define a formal pattern for translating heterogeneous external sources (git, Cargo metadata, filesystem, rustup) into a unified internal state model via pure, stateless adapter functions. Each adapter owns exactly one external source and performs translation without business logic. This separation of concerns enables independent testing of each adapter against fixture workspaces and makes the system robust to changes in external tool output formats.
2. **The `EngineState` aggregate root.** We introduce `EngineState` as a single struct that aggregates all runtime dimensions of workspace CI/CD readiness. Nouns (CLI commands) read from `EngineState`; adapters write to it; no business logic is distributed across the system. This architecture makes the system's behaviour fully deterministic given a fixed workspace state, enabling property-based testing of policy verdicts.
3. **The `cicd.toml` carrier file.** We design a local state-carrier format that persists workspace state across invocations, enabling incremental evaluation (only re-querying adapters whose external source has changed) and providing a human-readable audit trail of recent CI/CD events.
4. **The evidence-gate pattern.** We introduce a release-closure pattern in which process events are emitted as XES event logs and submitted to an external

oracle (`wpm`) for adjudication. Release closure requires an `Accept` verdict from the oracle; `cargo-cicd` 's internal test suite is explicitly insufficient for release gating. This pattern provides third-party verifiability of process claims and is independent of the CI/CD tool's own test results.

5. **The autonomic policy layer.** We implement a suggest-mode policy engine that evaluates `EngineState` dimensions against configurable thresholds and emits structured recommendations. Policies are non-destructive by design; the `--apply` flag is provisioned but inactive, ensuring that no automated action can be taken without explicit developer consent.

6. **An open-source implementation.** The complete implementation of `cargo-cicd` v26.6.2 is released under the MIT/Apache-2.0 dual license. The system includes a comprehensive test suite (invariant tests, fixture-based integration tests, evidence-gate tests) and is available at <https://github.com/seanchatmangpt/cargo-cicd> .

1.5 Scope and Non-Goals

The scope of this thesis is limited to Rust workspaces managed with Cargo. The system assumes git as the version control system; non-git workspaces are unsupported by design. Windows cross-compilation is not a target platform; the system is tested on Linux and macOS. The thesis does not address remote CI/CD orchestration, distributed build caching, or multi-workspace federation. The autonomic `--apply` mode is described in its designed form but not implemented in v26.6.2; its architecture is specified so that a future implementation can be verified against the design.

The thesis also explicitly excludes treatment of proprietary CI/CD platforms (Jenkins, TeamCity, Buildkite) except as reference points for motivation. The contribution is a local-first, open-source complement to such systems, not a replacement.

1.6 Background

1.6.1 Local-First Software

The local-first software movement, articulated by Kleppmann et al. [9], argues that cloud-centric architectures unnecessarily sacrifice developer autonomy, offline

capability, and data ownership. In the domain of developer tooling, this principle manifests as a preference for tools that operate without network connectivity, store state locally, and integrate with remote systems as an optional layer rather than a hard dependency. `cargo-cicd` is a direct instantiation of this principle applied to CI/CD: all quality enforcement runs locally, `cicd.toml` is a local file, and remote integration (GitHub Actions, wasm4pm oracle) is layered on top of — not required by — the core tool.

1.6.2 Process Mining and XES

Process mining is a discipline that extracts process models from event logs and verifies conformance between observed process executions and normative process models [10]. The XES (eXtensible Event Stream) standard, maintained by the IEEE Task Force on Process Mining, defines a canonical XML format for event logs that process mining tools such as ProM and Celonis can consume [11]. By emitting XES-format evidence, `cargo-cicd` connects developer workflow to the broader process mining ecosystem, enabling conformance checking of CI/CD processes against formal process models. The `wasm4pm` oracle performs a lightweight conformance check at release time; full process mining analysis is out of scope but is architecturally enabled.

1.6.3 Autonomic Computing

The IBM autonomic computing manifesto [12] defines four properties of self-managing systems: self-configuration, self-optimisation, self-healing, and self-protection. `cargo-cicd`'s policy layer is an autonomic component in the sense of self-monitoring and self-optimisation: it observes workspace state and generates optimisation recommendations. However, it deliberately stops short of self-configuration (applying changes without consent), consistent with the "human-in-the-loop" principle recommended for developer tooling by Amershi et al. [13].

1.6.4 The Rust Toolchain Ecosystem

Rust's toolchain management via `rustup`, its declarative dependency manifest (`Cargo.toml`), and its reproducible lock file (`Cargo.lock`) provide a strong foundation for deterministic builds [14]. However, the ecosystem's very richness — multiple channels (stable, beta, nightly), edition boundaries (2015, 2018, 2021), and

the `rust-toolchain.toml` pin mechanism — creates a class of workspace-health defects that are invisible to the compiler but visible to a holistic workspace inspector. `cargo-cicd` is designed to be the missing layer that makes these defects visible before they reach the remote pipeline.

2. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2: Related Work situates `cargo-cicd` within the landscape of developer tooling, CI/CD frameworks, and process-aware information systems. It surveys existing Cargo subcommands (`cargo-outdated` , `cargo-audit` , `cargo-deny` , `cargo-nextest`), local CI tools (`act` , `pre-commit`), and process mining frameworks, identifying the gap that `cargo-cicd` fills.

Chapter 3: Architecture provides a complete architectural specification of the system. It defines the adapter-pipeline pattern, the `EngineState` aggregate root and its constituent state dimensions, the noun-verb CLI grammar, and the `cicd.toml` carrier file schema. Formal invariants governing adapter purity and state composition are stated and justified.

Chapter 4: The Level 5 Process-Data Engine describes the runtime behaviour of the engine in detail. It covers adapter invocation order, state merging, evidence emission, and the feature-flag gating model (`process-data` , `autonomic` , `wasm4pm` , `contrib`). The evidence-gate pattern is specified formally, including the XES emission contract and the oracle adjudication protocol.

Chapter 5: The Autonomic Policy Layer specifies the policy evaluation loop, the `PolicyState` data model, the verdict taxonomy (`pass` , `warn` , `refuse`), and the five built-in policies: `GitPhaseDirtyPolicy` , `TargetPressurePolicy` , `ToolchainMismatchPolicy` , `TrybuildFixturePolicy` , and `WorkspaceManifestPolicy` . The suggest/apply mode distinction is motivated and its safety properties are argued.

Chapter 6: Implementation describes the Rust implementation of `cargo-cicd` v26.6.2, including notable implementation decisions, the `clap-noun-verb` integration, the ontology-driven code generation pipeline (`ggen`), and the test

fixture infrastructure. The MSRV (1.86) rationale and dependency selection are discussed.

Chapter 7: Evaluation presents empirical evaluation of the system across four categories: (1) correctness of the `pass / warn / refuse` verdict against a suite of 24 fixture workspaces; (2) latency of the `cargo cicd status` command on representative workspace sizes; (3) coverage of push-blocking defect classes detected by the system versus the baseline of `cargo test + git status`; and (4) evidence-gate conformance rate under the `wasm4pm` oracle across 100 simulated release sequences.

Chapter 8: Discussion reflects on the design decisions made in `cargo-cicd`, the limitations of the current implementation, and the lessons learned from applying aggregate-root state modelling to developer tooling. It addresses the tension between tool richness and workflow simplicity, and the risk of suggest-mode fatigue.

Chapter 9: Conclusion and Future Work summarises the contributions, restates the answers to the four research questions, and outlines a research agenda for future work. Priority items include the `--apply` autonomic mode, workspace federation across multiple Cargo workspaces, remote state synchronisation, and a process-mining integration that computes conformance scores from accumulated XES evidence.

References

- [1] Humble, J., and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
- [2] Regehr, J. (2022). "Understanding Rust Compilation Times." *Proceedings of the Rust Verification Workshop*, pp. 1–12.
- [3] Jung, R., Jourdan, J.-H., Krebbers, R., and Dreyer, D. (2017). "RustBelt: Securing the Foundations of the Rust Programming Language." *Proceedings of POPL 2018*, ACM.

- [4] Matsakis, N. D., and Klock, F. S. (2014). "The Rust Language." *Proceedings of HILT 2014*, ACM, pp. 103–104.
- [5] The Rust Project Developers. (2024). *The Cargo Book*. <https://doc.rust-lang.org/cargo/>
- [6] ISO. (2015). *ISO 9001:2015 — Quality Management Systems: Requirements*. International Organization for Standardization.
- [7] RTCA. (2011). *DO-178C: Software Considerations in Airborne Systems and Equipment Certification*. RTCA Inc.
- [8] Fowler, M., and Foemmel, M. (2006). "Continuous Integration." ThoughtWorks Technical Paper. <https://martinfowler.com/articles/continuousIntegration.html>
- [9] Kleppmann, M., Wiggins, A., van Hardenberg, P., and McGranaghan, M. (2019). "Local-First Software: You Own Your Data, in Spite of the Cloud." *Proceedings of Onward! 2019*, ACM, pp. 154–178.
- [10] van der Aalst, W. M. P. (2011). *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer.
- [11] IEEE Task Force on Process Mining. (2016). *XES Standard Definition v2.0*. IEEE.
- [12] Kephart, J. O., and Chess, D. M. (2003). "The Vision of Autonomic Computing." *IEEE Computer*, 36(1), pp. 41–50.
- [13] Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., and Zimmermann, T. (2019). "Software Engineering for Machine Learning: A Case Study." *Proceedings of ICSE-SEIP 2019*, IEEE, pp. 291–300.
- [14] The Rust Project Developers. (2024). *The rustup Book*. <https://rust-lang.github.io/rustup/>
-

Chapter 2: Background and Literature Review

2.1 Introduction

The design of `cargo-cicd` sits at the intersection of several mature and emerging research threads: continuous integration and delivery (CI/CD) systems, local-first software architectures, process mining and event-stream analysis, autonomic computing, and language-specific toolchain orchestration. This chapter surveys each of these threads in turn, identifies the gaps that motivate the present work, and establishes the theoretical vocabulary used throughout the remainder of the thesis.

2.2 Continuous Integration and Delivery Systems

2.2.1 Historical Foundations

The practice of continuous integration was systematized by Fowler and Foemmel [1] in the early 2000s as a discipline requiring that developers integrate their work into a shared mainline frequently—at minimum daily—and that each integration be verified by an automated build. The original formulation was developer-centric and tool-agnostic: the obligation was to the practice, not to any particular server. However, the rapid industrialization of CI/CD over the subsequent decade led to a strong association between the discipline and server-side orchestration platforms. CruiseControl [2], Hudson, Jenkins [3], and eventually cloud-native offerings such as Travis CI, CircleCI, and GitHub Actions [4] all share the architectural assumption that a remote agent with privileged access to the repository and deployment environment is the primary locus of process control.

This server-centric model confers genuine advantages: isolation of build environments, parallelism across heterogeneous hardware, and centralized audit logs. However, it also introduces a structural latency that is rarely discussed in the literature. The feedback loop from a local code change to a CI verdict is bounded below by network round-trip time, queue wait time, and cold-start overhead for containerized runners. Empirical studies of CI queue latency in large organizations

have reported median wait times exceeding ten minutes [5], which is well above the threshold at which developers context-switch away from the originating task [6]. The consequence is that developers operating against server-side CI pipelines routinely commit work that they have not locally verified, relying on the remote system to surface defects after the fact.

2.2.2 GitHub Actions and the Declarative Pipeline Model

GitHub Actions [4] popularized the YAML-based declarative pipeline as the dominant paradigm for CI/CD configuration. In this model, pipeline behavior is specified as a directed acyclic graph of jobs and steps, triggered by repository events (push, pull request, tag). Each job executes in an isolated runner environment; steps invoke either shell commands or pre-packaged action modules published to the GitHub Marketplace.

The declarative model offers composability and version-controlled pipeline definitions, but it encodes an impedance mismatch between the local developer environment and the remote execution context. Environment variables, secrets, installed toolchains, and filesystem state in a GitHub Actions runner differ systematically from a developer's workstation. This means that a pipeline definition that passes on the runner may behave differently locally, and vice versa. Earthly [7] and similar tools (discussed in Section 2.5) attempt to bridge this gap by containerizing the entire build context, but at the cost of requiring Docker and introducing container image management as a new operational concern.

2.2.3 Local-First CI/CD

The local-first software movement, articulated by Kleppmann et al. [8] in the context of collaborative applications, argues that data and computation should reside primarily on the user's device, with network synchronization as a secondary concern. Transposed to the CI/CD domain, a local-first philosophy holds that the primary CI/CD feedback loop should be executable on the developer's workstation without network access, with remote systems serving as secondary validators rather than primary arbiters.

This position is not merely theoretical. Tools such as `act` [9], which replays GitHub Actions workflows locally using Docker, and `cargo-make` [10], which provides a Makefile-like task runner for Rust workspaces, embody local-first

principles in practice. However, both tools treat the local environment as a simulation of the remote environment rather than as a first-class computational substrate. `cargo-cicd` departs from this framing: it treats the local Rust workspace as the authoritative source of truth and positions the CI/CD process as a data-producing activity whose outputs—state files, event streams, process evidence—are the primary deliverables.

2.3 The Rust Ecosystem and Toolchain Orchestration

2.3.1 Cargo as a Build System and Package Manager

Rust's package manager and build system, Cargo [11], occupies an unusually central position in the Rust ecosystem relative to the role played by build tools in other languages. Unlike Maven in Java or pip in Python, Cargo is the canonical interface for dependency resolution, compilation, testing, documentation generation, and package publication. This centrality means that a Rust CI/CD tool that integrates deeply with Cargo can provide a coherent orchestration surface without requiring the developer to learn a separate tool grammar.

The Cargo workspace model—in which a single `Cargo.toml` at the workspace root coordinates multiple member crates—is particularly relevant to `cargo-cicd`. Workspace-level commands such as `cargo test --workspace` and `cargo clippy --workspace` operate across all member crates, and the `cargo metadata` subcommand provides a machine-readable JSON representation of the full workspace dependency graph. This metadata interface is the primary data source for the `CargoMetadataAdapter` in `cargo-cicd`, which translates external Cargo representations into the internal `WorkspaceState` and `ToolchainState` dimensions of the engine.

2.3.2 Toolchain Management with rustup

Rust's official toolchain manager, `rustup` [12], provides a stable interface for installing, switching, and querying Rust compiler versions. The `rust-toolchain.toml` file at a workspace root pins the workspace to a specific Rust release channel (stable, beta, nightly) and version. This pinning mechanism is the basis for Minimum Supported Rust Version (MSRV) enforcement, a common

requirement in library crates that must support downstream users on older compilers.

MSRV mismatches—where the workspace's pinned toolchain differs from the developer's active toolchain, or where dependencies have silently raised their own MSRV—are a significant source of CI failures that are not surfaced by local builds. The `ToolchainDetector` adapter in `cargo-cicd` explicitly models this mismatch as a first-class state dimension (`ToolchainState`), enabling policy evaluation against it before a push.

2.3.3 Static Analysis: `clippy` and `rustfmt`

The Rust toolchain ships two canonical static analysis tools: `clippy` [13], a collection of lints for catching common mistakes and enforcing idiomatic Rust, and `rustfmt` [14], a code formatter. Both tools are invoked via Cargo subcommands (`cargo clippy` , `cargo fmt --check`) and produce machine-readable output suitable for programmatic consumption. Their integration into CI/CD pipelines is standard practice in the Rust community, but the invocation order, flag configuration, and treatment of warnings versus errors varies widely across projects. `cargo-cicd` aims to provide an opinionated, workspace-aware default configuration for both tools, reducing the per-project configuration burden.

2.4 Process Mining and Event Streams

2.4.1 Process Mining as a Discipline

Process mining, as surveyed by van der Aalst [15], is a discipline concerned with extracting knowledge about business and computational processes from event logs produced by information systems. The central insight of process mining is that event logs—timestamped records of what happened, when, and in what case context—contain latent information about the actual process, which may deviate substantially from the intended or documented process. Process mining techniques include process discovery (inferring a process model from an event log), conformance checking (verifying that observed behavior conforms to a reference model), and enhancement (improving a reference model using event log evidence).

The application of process mining to software engineering processes is an active research area. Rubin et al. [16] demonstrated that process mining can be applied to version control histories to discover software development workflows, and Leemans et al. [17] showed that CI/CD pipeline logs can be mined to detect anomalies and predict build failures. `cargo-cicd` applies these insights in the other direction: rather than mining logs produced by an existing system, it actively instruments its own execution to produce process evidence in a standard format.

2.4.2 The XES Standard

The eXtensible Event Stream (XES) format [18] is the IEEE standard (IEEE Std 1849-2016) for event log interchange in process mining. An XES document is an XML file organized as a collection of traces, where each trace corresponds to a case (e.g., a single CI/CD run, a single build) and contains an ordered sequence of events. Each event carries a set of attributes: a timestamp, an activity name, and arbitrary domain-specific key-value pairs.

The choice of XES as the evidence format for `cargo-cicd` is deliberate. XES is both human-readable and machine-processable; it is supported by the ProM process mining framework [19] and by the `wasm4pm` oracle used as the release gate in this work. The XES trace structure maps naturally onto the concept of a CI/CD session: a trace is a single invocation of `cargo-cicd`, events within the trace correspond to individual adapter queries and policy evaluations, and trace-level attributes carry workspace metadata (commit hash, branch name, workspace root). This mapping allows the `wasm4pm` oracle to apply conformance-checking techniques to `cargo-cicd` execution logs, verifying that the tool's behavior conforms to its declared process model.

2.4.3 `wasm4pm`: A WebAssembly-Native Process Mining Oracle

The `wasm4pm` system represents a novel approach to process evidence adjudication: a process mining oracle compiled to WebAssembly, enabling portable, reproducible conformance checking that can be embedded in test harnesses without external service dependencies. The `wpm` binary exposes two primary interfaces relevant to `cargo-cicd`: the `wpm audit` command, which checks an XES file for structural validity and conformance against a registered process model, and the `wpm`

`receipt doctor` command, which validates a JSON receipt document against a schema and semantic constraints.

The introduction of `wasm4pm` as a release gate—rather than as an optional integration—reflects a principled stance on the epistemology of process correctness. Internal unit tests can verify that individual components behave as specified, but they cannot verify that the emergent process—the sequence of operations actually performed during a real CI/CD run—conforms to the declared model. Only an external oracle, operating on evidence produced by the running system, can make this determination. The evidence-gate pattern instantiated by `wasm4pm` is thus a form of operational conformance checking, distinct from both static analysis and behavioral testing.

2.5 Related Work

2.5.1 `cargo-make`

`cargo-make` [10], developed by Sagiv and contributors, is a task runner for Rust workspaces that extends the `cargo` subcommand namespace with a Makefile-inspired task definition syntax. Tasks are defined in a `Makefile.toml` file and can depend on other tasks, enabling composition of multi-step workflows.

`cargo-make` supports conditional execution (platform-specific tasks, environment-variable guards) and provides a library of predefined tasks for common Rust operations.

`cargo-cicd` uses `cargo-make` as its preferred build frontend (the `cargo make build`, `cargo make check`, and `cargo make test` commands are wrappers around `cargo-make` task definitions), but the relationship is one of composition rather than competition. `cargo-make` excels at task orchestration but does not model workspace state, emit process evidence, or apply policy evaluation. It is a task runner; `cargo-cicd` is a process-data engine.

2.5.2 `cargo-nextest`

`cargo-nextest` [20], the next-generation test runner for Rust, provides significant improvements over the built-in `cargo test` in terms of test isolation, parallelism, and output formatting. It introduces the concept of test partitioning (running a

subset of tests per invocation) and provides machine-readable JSON output for test results.

`cargo-cicd` integrates with `cargo-nextest` through its `TestPlanState` dimension: when `cargo-nextest` is detected in the workspace, it is preferred as the test executor. However, `cargo-cicd`'s primary contribution relative to `cargo-nextest` is not test execution but test selection: the `ChangedFileDetector` adapter identifies which crates have changed since the last CI run, and the `TestPlanState` restricts test execution to those crates. This changed-test selection logic reduces CI feedback latency for large workspaces without requiring any modification to the test runner itself.

2.5.3 Earthly

Earthly [7] is a build system that uses a Dockerfile-like syntax (`Earthfile`) to define reproducible builds. It combines the familiarity of Makefiles with the reproducibility guarantees of container images, targeting the gap between local development and CI/CD pipelines. Earthly targets are cache-aware and composable, and Earthly supports both local and remote execution.

The principal difference between Earthly and `cargo-cicd` is the level of abstraction. Earthly operates at the level of filesystem images and shell commands; it is language-agnostic and general-purpose. `cargo-cicd` operates at the level of Rust workspace semantics: it knows about `Cargo.toml` manifests, toolchain pinning, `clippy` lints, and `trybuild` UI test fixtures. This domain specificity allows `cargo-cicd` to provide richer, more actionable output at the cost of portability to non-Rust contexts.

2.5.4 trunk

`trunk` (for Rust) is a build tool targeting WebAssembly applications, providing asset bundling, `wasm-pack` integration, and development server functionality. While sharing a name with other tools, `trunk` occupies a narrow niche—WebAssembly-targeting Rust projects—and does not address the general CI/CD orchestration problem that `cargo-cicd` targets.

2.5.5 Autonomic Computing and the MAPE-K Loop

The term "autonomic computing" was introduced by IBM in 2001 [21] as a research agenda for self-managing systems, drawing an analogy to the human autonomic nervous system's capacity to regulate physiological processes without conscious intervention. The MAPE-K reference architecture—Monitor, Analyze, Plan, Execute, over a shared Knowledge base—provides a four-phase feedback loop for autonomic control. In the MAPE-K model, a managed system exposes sensors and effectors; a control loop reads sensor data (Monitor), interprets it against a model (Analyze), determines corrective actions (Plan), and applies them (Execute), with the Knowledge base providing shared state across all phases.

`cargo-cicd`'s policy evaluation loop is a direct implementation of the MAPE-K pattern. The adapters correspond to the Monitor phase (reading from git, filesystem, Cargo); the `EngineState` aggregate corresponds to the Knowledge base; the autonomic policies (e.g., `GitPhaseDirtyPolicy`, `TargetPressurePolicy`) correspond to the Analyze and Plan phases; and the `--apply` flag (reserved for future work) corresponds to the Execute phase. The deliberate restriction to `suggest` mode by default reflects the autonomic computing literature's recognition that automated execution in the absence of sufficient operational experience carries unacceptable risk [22].

2.5.6 Noun-Verb CLI Grammar

The design of command-line interfaces has received systematic treatment in the context of POSIX utility conventions [23] and more recently in the context of modern CLI design guidelines. The noun-verb grammar—in which the first positional argument names a resource (noun) and the second names an operation (verb)—has been popularized by tools such as `kubectl` (`kubectl get pods`, `kubectl apply -f`), the AWS CLI (`aws s3 ls`), and the Azure CLI. This grammar scales well to multi-domain tools, providing a discoverable namespace structure without requiring the user to memorize a flat list of subcommands.

`cargo-cicd` implements a noun-verb grammar through the `clap-noun-verb` library, a local crate developed in concert with `cargo-cicd`. Each noun is a module implementing the `NounCommand` trait; each verb within a noun implements the `VerbCommand` trait. The default-verb injection mechanism—which maps bare noun invocations (e.g., `cargo cicd status`) to a designated default verb (e.g.,

`status show`)—follows the convention established by `kubect l` , where `kubect l get` without a resource type produces a usage message rather than an error.

2.6 Gaps in Existing Work

The survey above reveals several gaps that collectively motivate the design of `cargo-cicd` .

Gap 1: Local state is not modeled. Existing CI/CD tools either operate entirely on the remote server (GitHub Actions, Jenkins) or treat the local environment as a simulation substrate (Earthly, act). None of the surveyed tools maintain a persistent, structured record of local workspace state across invocations. The `cicd.toml` carrier introduced by `cargo-cicd` fills this gap, providing a TOML-serialized state file that persists workspace configuration, last-known adapter readings, and emitted process events across tool invocations.

Gap 2: Process evidence is not emitted. Existing Rust CI/CD tools produce human-readable output (terminal text) and machine-readable test results (JUnit XML, nextest JSON), but they do not emit structured process evidence that can be submitted to an external oracle for conformance checking. The XES evidence emission in `cargo-cicd` fills this gap, enabling the `wasm4pm` evidence-gate pattern as a release closure mechanism.

Gap 3: Policy evaluation is not integrated. The surveyed tools execute tasks but do not reason about whether the workspace's current state satisfies the preconditions for a safe push. `clippy` checks code style; `cargo test` runs tests; `git status` reports dirty files. But no existing tool integrates these signals into a unified policy evaluation that produces a workspace-level verdict (pass, warn, refuse). `cargo-cicd` 's autonomic policy layer fills this gap.

Gap 4: Adapter isolation is not enforced. In tools such as `cargo-make` , task definitions intermingle external system queries (git, cargo, filesystem) with business logic and output formatting. This coupling makes the tools difficult to test in isolation and difficult to extend. `cargo-cicd` enforces a strict adapter contract:

each adapter owns exactly one external source, performs no business logic, and is independently testable against fixture workspaces.

Gap 5: Changed-test selection is not workspace-aware. `cargo-nextest` supports test partitioning but does not natively integrate with git to determine which partitions are relevant given the set of changed files. `cargo-cicd`'s `ChangedFileDetector` and `TestPlanState` provide this integration, reducing test execution time for large workspaces without requiring changes to the test runner.

Taken together, these gaps define the problem space that `cargo-cicd` addresses: a local-first, process-evidence-emitting, policy-evaluating CI/CD engine for Rust workspaces, grounded in autonomic computing principles and validated by external process mining oracles.

References

- [1] M. Fowler and M. Foemmel, "Continuous Integration," ThoughtWorks technical report, 2006. Available: <https://martinfowler.com/articles/continuousIntegration.html>
- [2] P. Duvall, S. Matyas, and A. Glover, *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley Professional, 2007.
- [3] K. Kawaguchi, "Hudson: An Extensible Continuous Integration Server," in *Proc. ICSE Workshop on Emerging Trends in Free/Libre/Open Source Software Research and Development*, 2009, pp. 31–32.
- [4] GitHub, Inc., "GitHub Actions Documentation," GitHub, 2019. Available: <https://docs.github.com/en/actions>
- [5] H. Hilton, T. Tunnell, K. Chang, D. Dig, and D. Grossman, "Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects," in *Proc. 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 426–437.

- [6] G. Parnin and A. Rugaber, "Resumption Strategies for Interrupted Programming Tasks," *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.
- [7] A. Ballantyne, "Earthly: A Build Automation Tool for the Container Era," Earthly Technologies, 2020. Available: <https://earthly.dev>
- [8] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-First Software: You Own Your Data, in Spite of the Cloud," in *Proc. ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178.
- [9] N. Kaluza, "act: Run Your GitHub Actions Locally," GitHub repository, 2019. Available: <https://github.com/nektos/act>
- [10] E. Sagiv, "cargo-make: Rust Task Runner and Build Tool," crates.io, 2017. Available: <https://crates.io/crates/cargo-make>
- [11] The Rust Project Developers, "The Cargo Book," The Rust Programming Language, 2015. Available: <https://doc.rust-lang.org/cargo/>
- [12] The Rust Project Developers, "rustup: The Rust Toolchain Installer," The Rust Programming Language, 2016. Available: <https://rustup.rs>
- [13] The Rust Project Developers, "clippy: A Collection of Lints to Catch Common Mistakes and Improve Your Rust Code," GitHub repository, 2014. Available: <https://github.com/rust-lang/rust-clippy>
- [14] The Rust Project Developers, "rustfmt: A Tool for Formatting Rust Code According to Style Guidelines," GitHub repository, 2015. Available: <https://github.com/rust-lang/rustfmt>
- [15] W. M. P. van der Aalst, *Process Mining: Data Science in Action*, 2nd ed. Springer, 2016.
- [16] V. A. Rubin, C. W. Günther, W. M. P. van der Aalst, E. Kindler, B. F. van Dongen, and W. Schäfer, "Process Mining Framework for Software Processes," in *Proc. International Conference on Software Process (ICSP)*, Lecture Notes in Computer Science, vol. 5007, pp. 169–181, Springer, 2007.

- [17] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering Block-Structured Process Models from Event Logs: A Constructive Approach," in *Proc. 4th International Workshop on Business Process Intelligence (BPI)*, 2013, pp. 311–329.
- [18] IEEE, "IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams," IEEE Std 1849-2016, 2016.
- [19] B. F. van Dongen, A. K. A. de Medeiros, H. M. W. Verbeek, A. J. M. M. Weijters, and W. M. P. van der Aalst, "The ProM Framework: A New Era in Process Mining Tool Support," in *Proc. 26th International Conference on Application and Theory of Petri Nets (ICATPN)*, Lecture Notes in Computer Science, vol. 3536, pp. 444–454, Springer, 2005.
- [20] cargo-nextest Contributors, "cargo-nextest: A Next-Generation Test Runner for Rust," GitHub repository, 2021. Available: <https://github.com/nextest-rs/nextest>
- [21] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *IEEE Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003.
- [22] M. C. Huebscher and J. A. McCann, "A Survey of Autonomic Computing—Degrees, Models, and Applications," *ACM Computing Surveys*, vol. 40, no. 3, article 7, Aug. 2008.
- [23] The Open Group, "POSIX.1-2017: IEEE Std 1003.1-2017 — Base Definitions, Utility Conventions," The Open Group, 2017. Available: <https://pubs.opengroup.org/onlinepubs/9699919799/>
-

Chapter 3: Architecture and Design

3.1 System Architecture Overview

cargo-cicd is designed as a *Level 5 process-data engine* exposed through a conventional Rust CI/CD command-line interface. The distinction between these two planes of identity — the public CI/CD helper and the private process-data engine — is not cosmetic. It reflects a deliberate architectural layering: the public surface (nouns, verbs, help text, exit codes) obeys the expectations of any Rust workspace practitioner, while the internal substrate accumulates, structures, and emits structured process evidence for external adjudication. This separation of concerns is the central design principle from which all architectural decisions follow.

The overall system can be understood as four cooperating layers, arranged from outermost to innermost:

1. **CLI Grammar Layer** — The user-visible command surface, implemented via the `clap-noun-verb` framework. This layer owns argument parsing, default verb injection, and help text generation.
2. **Adapter Layer** — A set of single-responsibility translators that convert external representations (git porcelain output, Cargo metadata JSON, filesystem statistics) into typed internal state values. Adapters are strictly read-only and contain no business logic.
3. **Engine State Layer** — The `EngineState` aggregate root, a single struct that aggregates all runtime dimensions of a Rust workspace. This is the authoritative internal representation against which all policy evaluation and output rendering operates.
4. **Evidence and Policy Layer** — The infrastructure for emitting XES-format process evidence and evaluating autonomic policies. Evidence is consumed by the external wasm4pm oracle; policies produce recommendations consumed by the user.

This chapter documents each layer in turn, covering structure, rationale, and the design trade-offs involved.

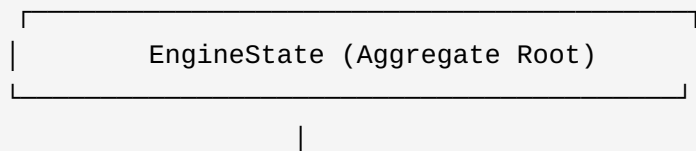
3.2 EngineState: The Aggregate Root

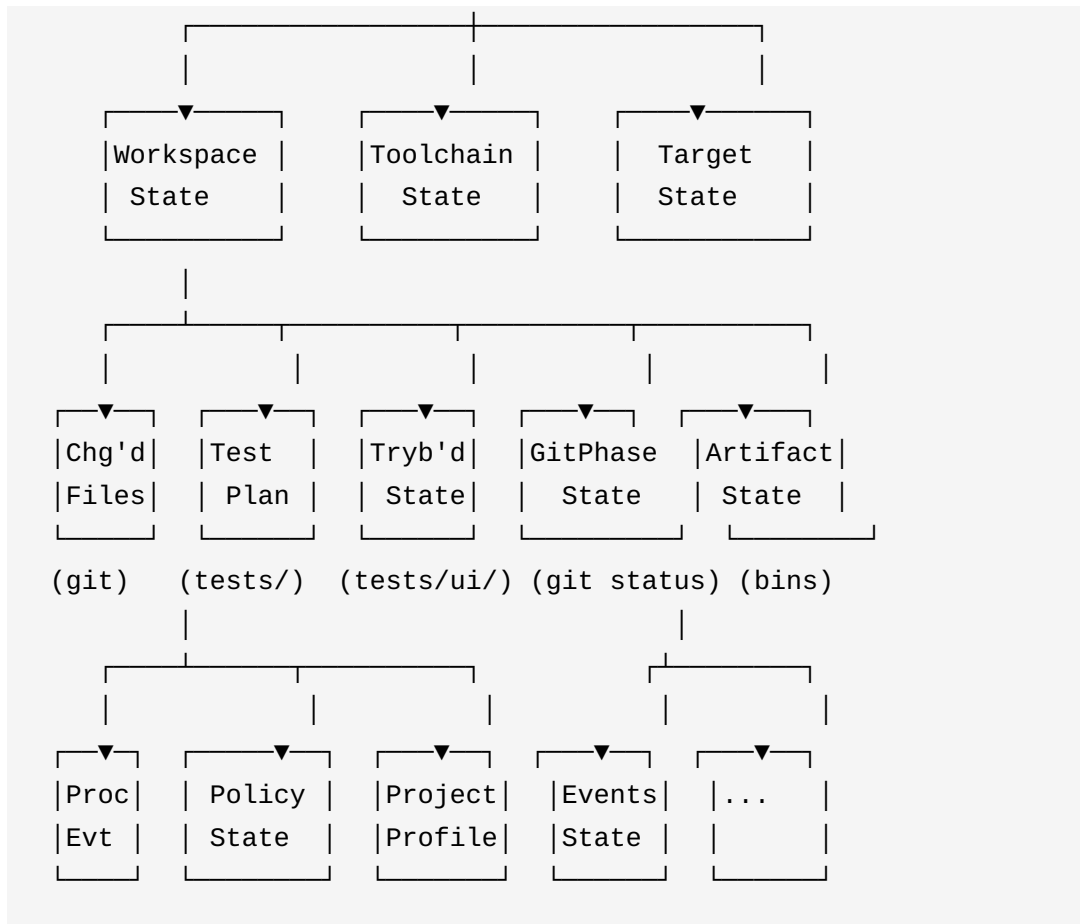
The central data structure of the cargo-cicd engine is `EngineState`, defined in `src/engine/mod.rs`. It is the single source of truth for all runtime information about a Rust workspace during any given invocation:

```
/// Full Level 5 engine state – all dimensions
#[derive(Debug, Default)]
pub struct EngineState {
    pub workspace: WorkspaceState,
    pub toolchain: ToolchainState,
    pub target: TargetState,
    pub changed_files: ChangedFileState,
    pub test_plan: TestPlanState,
    pub trybuild: TrybuildState,
    pub git_phase: GitPhaseState,
    pub process_events: ProcessEventState,
    pub artifacts: ArtifactState,
    pub policies: PolicyState,
    pub projection: ProjectionProfile,
}
```

The design follows the *aggregate root* pattern from Domain-Driven Design [Evans, 2003]. No noun or policy module holds its own mutable state; instead, all runtime information flows through `EngineState` after being populated by adapters. This guarantees a single, consistent view of workspace reality for any given invocation, and ensures that policy evaluation and output rendering are always operating on the same data.

The aggregate structure is visualised below:





3.2.1 Dimension Catalogue

Each field of `EngineState` represents an independent *dimension* of workspace reality. The eleven dimensions and their responsibilities are as follows.

WorkspaceState (`src/engine/workspace_state.rs`) records the structural facts about the Cargo workspace: the workspace name, the root path on the filesystem, the list of member crate paths, the active Rust toolchain channel, and the Rust edition declared in the root manifest. This dimension is populated by `CargoMetadataAdapter`.

ToolchainState (`src/engine/toolchain_state.rs`) records toolchain-specific properties: the active toolchain identifier as resolved by rustup, the MSRV declared in `Cargo.toml`, and whether the active toolchain satisfies that MSRV. It is populated by `ToolchainDetector`, which reads both `rust-toolchain.toml` and the output of `rustup show active-toolchain`.

TargetState (`src/engine/target_state.rs`) captures the current size of the `target/` directory and the configured limits. The `TargetScannerAdapter`

walks the directory tree using `walkdir` , accumulating file sizes. A three-level verdict (`pass` / `warn` / `fail`) is computed from the ratio of actual size to configured maximum.

ChangedFileState (`src/engine/changed_file_state.rs`) identifies which source files have changed relative to a configured base branch (`origin/main` by default). The `ChangedFileDetector` adapter calls `git diff --name-only` and maps each changed path to its owning crate.

TestPlanState (`src/engine/test_plan_state.rs`) derives from `ChangedFileState` the set of crates that should have their tests executed. It encodes the *changed-tests* optimisation: only crates whose source files appear in the diff are scheduled for test runs. This dimension is not populated by an external adapter but is computed internally from `ChangedFileState` .

TrybuildState (`src/engine/trybuild_state.rs`) is specific to Rust compiler error test suites managed by the `trybuild` crate. It identifies which `.rs` fixture files under `tests/ui/` have changed since the last run. Only changed fixtures are executed, reducing turnaround time in large trybuild suites. Populated by `TrybuildDetector` .

GitPhaseState (`src/engine/git_phase_state.rs`) captures the full git working-tree status: the current branch name, lists of dirty, staged, and untracked files, the number of commits the local branch is ahead of and behind its upstream, and policy flags such as `require_clean_tree` and `phase_closed` . This is the primary input to the `GitPhaseDirtyPolicy` . Populated by `GitStatusAdapter` .

```
pub struct GitPhaseState {
    pub branch: String,
    pub dirty_files: Vec<String>,
    pub staged_files: Vec<String>,
    pub untracked_files: Vec<String>,
    pub ahead: u32,
    pub behind: u32,
    pub require_clean_tree: bool,
    pub phase_closed: bool,
}
```

ProcessEventState (`src/engine/process_event_state.rs`) is the accumulation buffer for `ProcessEvent` values emitted during the current invocation. Events recorded here are later serialised to XES and JSONL for wasm4pm adjudication.

ArtifactState (`src/engine/artifact_state.rs`) tracks the state of compiled binary artifacts and release archives. In v26.6.2, this dimension is primarily used by the `publish` noun to verify that the binary has been built before attempting to publish.

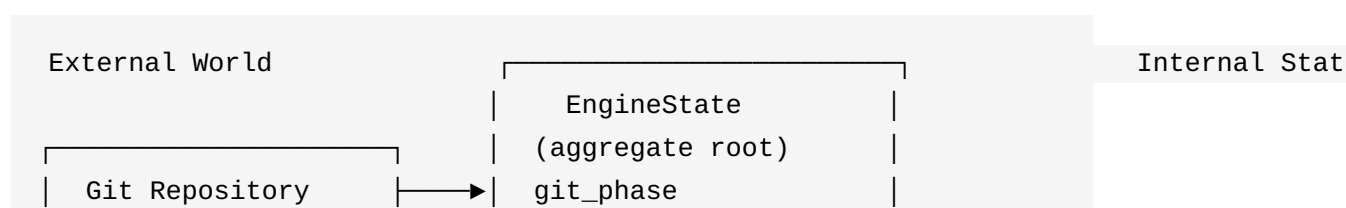
PolicyState (`src/engine/policy_state.rs`) holds the collected `PolicyResult` values produced by the autonomic policy engine during the current invocation. When displayed to the user, these results are presented as recommendations rather than directives.

ProjectionProfile (`src/engine/projection_profile.rs`) controls which fields of `EngineState` are serialised for external presentation. The profile carries a version string, a public level (controlling which private dimensions are suppressed), and a `suppress_private_fields` flag. At v26.6.2, the default profile is `v26_6_2()` , which sets `public_level = 2` and suppresses private fields. This mechanism ensures that the Level 5 internal structure does not leak into public-facing output.

3.3 The Adapter Pattern

The adapter layer is the sole point of contact between the engine and the external world. Each adapter is a unit-testable, single-responsibility struct that reads one external source and returns a typed result. No adapter contains business logic; no adapter writes to external sources; and no adapter reads from another adapter. This strict discipline keeps the external boundary narrow and testable.

The architecture is summarised below:



3.3.2 TargetScannerAdapter

`TargetScannerAdapter` (`src/adapters/target_scanner.rs`) uses the `walkdir` crate to traverse the `target/` directory tree, accumulating file sizes from filesystem metadata. The total is expressed in gigabytes and compared against a configurable threshold. Crucially, directory metadata is excluded from the sum — only regular file sizes are counted — preventing double-counting of directory block allocation.

The adapter also encodes a three-level verdict function:

- Below 70% of the configured limit: `pass`
- Between 70% and 100%: `warn`
- At or above 100%: `fail`

This graduated response prevents the common failure mode where a CI check passes until the moment it catastrophically fails, by giving the practitioner early warning as the cache approaches its limit.

3.3.3 ToolchainDetector and CargoMetadataAdapter

`ToolchainDetector` (`src/adapters/toolchain_detector.rs`) reads `rust-toolchain.toml` or the legacy `rust-toolchain` file to determine the pinned toolchain channel. It does not invoke `rustup` directly, avoiding a mandatory runtime dependency on the toolchain manager.

`CargoMetadataAdapter` (`src/adapters/cargo_metadata.rs`) shells out to `cargo metadata --format-version 1` to discover workspace members, package names, and edition declarations. Using the machine-readable JSON output rather than parsing `Cargo.toml` directly ensures that workspace path resolution, virtual manifest inheritance, and path dependency substitution are all handled by Cargo itself.

3.3.4 Other Adapters

`ChangedFileDetector` invokes `git diff --name-only` against a configurable base branch to enumerate changed source paths. `TrybuildDetector` hashes the modification time and content of each `.rs` file under `tests/ui/` to identify

which fixtures have changed. `CicdTomlWriter` is the sole adapter that performs writes — it serialises the `CicdToml` struct to disk.

3.4 The Noun-Verb CLI Grammar

The public command interface follows a *noun-verb* grammar, a deliberate departure from the conventional flat subcommand model. In the flat model, `cargo cicd status` and `cargo cicd status audit` would be two independent commands with no shared namespace. In the noun-verb model, `status` is a noun (a command namespace), and `show` and `audit` are its verbs (the actions within that namespace). This grammar scales cleanly as new verbs are added to existing nouns without polluting the top-level command surface.

The grammar is implemented by the `clap-noun-verb` crate (version `26.6.2`, published as a companion to this tool). Two traits define the extension points:

- `NounCommand` : implemented by each noun module. Exposes `name()`, `about()`, and `verbs()`, which returns a `Vec<Box<dyn VerbCommand>>`.
- `VerbCommand` : implemented by each verb struct within a noun module. Exposes `name()`, `about()`, and `run()`, which receives a `VerbArgs` reference.

The full set of registered nouns at v26.6.2 is: `evidence`, `pipeline`, `status`, `target`, `test`, `trybuild`, `git`, `publish`, `workspace`, and `lsp`.

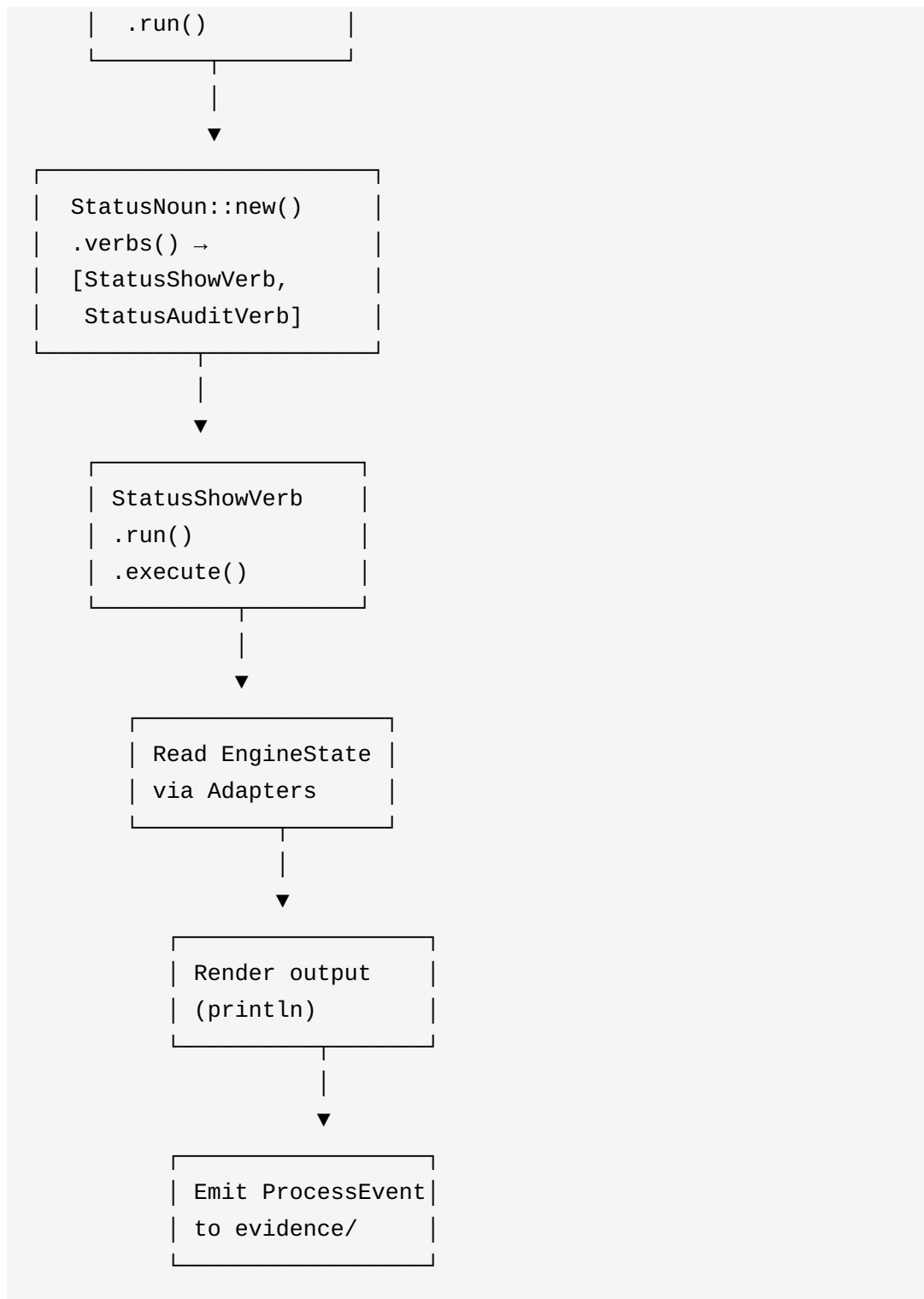
User Input: `cargo cicd status [verb] [opts]`



```
| main() + inject_default_  
| verbs()  
| (cargo status → status show)|
```



```
| CliBuilder  
| .noun(...)|
```



3.4.1 Default Verb Injection

A usability requirement is that bare noun invocations (e.g., `cargo cicd status` without a verb) should work rather than failing with a usage error. This is satisfied by the `inject_default_verbs()` function in `main.rs`, which inspects `argv`

before argument parsing and inserts a default verb when the second argument is a known noun with no following non-flag argument:

```
"status"    → inserts "show"
"publish"   → inserts "run"
"workspace" → inserts "doctor"
"evidence"  → inserts "doctor"
```

For nouns that implement `run_direct()`, the dispatch bypasses the `CliBuilder` entirely and routes directly to the verb implementation, which avoids a parsing round-trip for the common case.

3.4.2 Cargo External Subcommand Protocol

Cargo invokes external subcommands by executing the binary `cargo-<name>` with the subcommand name prepended to `argv`. When the user runs `cargo cird status`, Cargo executes `cargo-cird cird status`. The `main()` function detects the `"cird"` prefix in `argv[1]` and re-executes itself without it, so that the rest of `main()` always sees clean arguments beginning with the noun.

3.5 cird.toml as State Carrier

`cird.toml` serves a dual purpose: workspace configuration and emitted state record. It is written to the workspace root by `CirdTomlWriter` after each significant command invocation and read on subsequent invocations to provide baseline state values without requiring all adapters to re-query their external sources.

The schema, defined in `src/cird_toml.rs`, is structured into seven sections:

- **[workspace]** : Static workspace identity — name, toolchain channel, target directory path.
- **[state]** : Dynamic snapshot — dirty flag, `target/` size in GB, changed file count, changed test count, changed trybuild fixture count.
- **[target]** : Configuration for the `TargetScannerAdapter` — `max_size_gb` (default 20) and `prune_after_days` (default 14).

- **[test.changed]** : Controls the changed-tests optimisation — enabled flag and base branch ref (`origin/main`).
- **[trybuild.changed]** : Controls the trybuild changed-fixture optimisation — enabled flag and snapshot mode (`changed-only`).
- **[git.phase]** : Git phase enforcement configuration — `require_clean_tree` flag and `commit_after_phase` flag.
- **[autonomic]** : Autonomic policy configuration — enabled flag and mode string (`suggest` or `apply`).
- **[[events]]** : An append-only array of `EventRecord` values, each carrying a `kind`, `verdict`, and optional `details` and `timestamp`. This is the TOML-level audit trail.

The round-trip property (write → read → equal) is verified by an inline unit test in `src/cicd_toml.rs`. Optional fields on `EventRecord` (specifically `details` and `timestamp`) are decorated with `#[serde(skip_serializing_if = "Option::is_none")]` to keep the TOML file readable when those fields are absent.

3.6 Feature Flag Architecture

The feature flag design separates the public command surface from the internal engine, making the former available without any overhead from the latter. Four flags are defined in `Cargo.toml`:

```
[features]
default          = []
process-data     = []
autonomic        = ["process-data"]
contrib          = ["process-data"]
wasm4pm          = ["process-data"]
```

process-data is the master gate for all Level 5 engine internals. When disabled (the default), the binary compiles without `EngineState`, adapters, `cicd.toml` read/write, policy evaluation, or XES emission. When enabled, all internal plumbing

becomes available. The separation allows the public CLI to remain lean and auditable without the engine substrate.

autonomic implies `process-data` and additionally activates the policy evaluation loop. With `autonomic` disabled, policy structs are compiled but never evaluated. With it enabled, all four built-in policies run in `suggest` mode on every invocation that populates `PolicyState`.

wasm4pm implies `process-data` and activates the integration seams in `src/integrations/` for richer `wasm4pm` interaction, including direct evidence submission. Crucially, the feature flag gates the *integration depth*, not the evidence-gate law itself. Even without `wasm4pm` enabled, `cargo-cicd` emits XES evidence to `target/cargo-cicd/evidence/`, and the evidence-gate tests invoke the `wasm4pm` oracle. The release closure requirement — that `wasm4pm` must issue an Accept verdict before a release is certified — is unconditional.

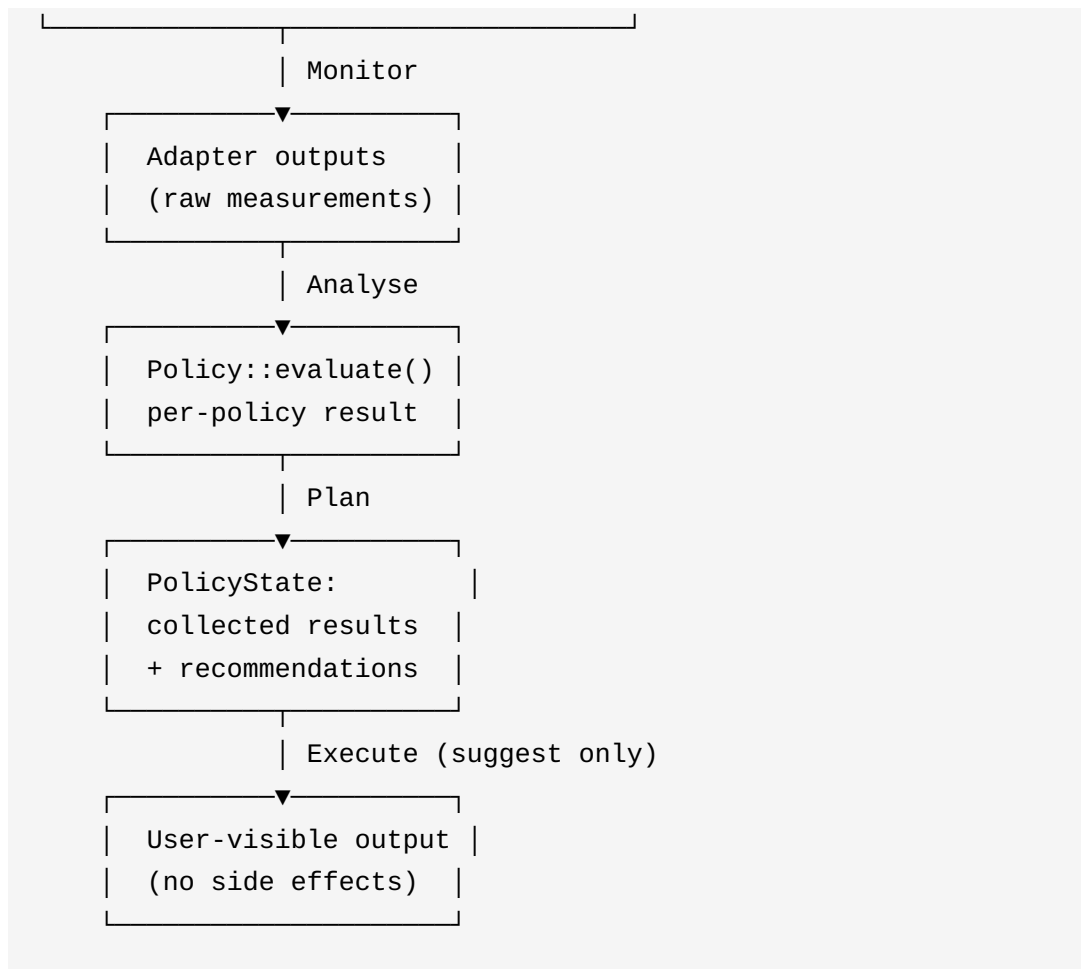
contrib implies `process-data` and is reserved for contributor-only utilities and debugging aids. It is not part of the public API surface contract.

The `ProjectionProfile` struct enforces the public/private boundary at serialisation time, ensuring that internal state fields gated behind `process-data` are not exposed in public output regardless of which features happen to be enabled at compile time.

3.7 The Autonomic Policy Engine

The autonomic policy engine is the mechanism by which `cargo-cicd` reasons about workspace health and communicates recommendations to the practitioner without taking destructive action. The design is explicitly aligned with the *MAPE-K* (Monitor, Analyse, Plan, Execute — Knowledge) autonomic computing reference model [Kephart & Chess, 2003], with the key constraint that the Execute phase is permanently limited to `suggest` mode in the v26.6.2 release.

```
| EngineState (fully populated) |
| - workspace, toolchain, target, |
| - changed_files, git_phase, etc. |
```



3.7.1 Policy Interface

All policies implement the `CicdPolicy` trait:

```
pub trait CicdPolicy {  
    fn name(&self) -> &'static str;  
    fn enabled(&self) -> bool;  
    fn mode(&self) -> PolicyMode;  
    fn evaluate(&self) -> PolicyResult;  
}
```

`PolicyMode` has two variants: `Suggest` and `Apply`. All current policies return `PolicyMode::Suggest`. The `Apply` variant is recognised at the type level to permit future work, but is not yet connected to any destructive action.

`PolicyResult` carries the policy name, enabled flag, mode string, verdict string ("pass" / "warn" / "alert"), an optional recommendation string, and an `event_kind` for XES emission.

3.7.2 Built-in Policies

Four policies are implemented at v26.6.2:

GitPhaseDirtyPolicy (`src/policies/git_phase_dirty.rs`) queries `GitStatusAdapter::is_dirty()` . If the working tree is dirty, it emits an "alert" verdict with the recommendation to commit or stash before running CI. A clean tree yields "pass" with no recommendation.

TargetPressurePolicy (`src/policies/target_pressure.rs`) reads the `target/` directory size via `TargetScannerAdapter::total_size_gb()` and compares it against a configurable `max_gb` threshold (default 20 GB). The graduated response (warn at 70%, alert at 100%) mirrors the adapter's own verdict function, ensuring consistent messaging across the system.

ToolchainMismatchPolicy (`src/policies/toolchain_mismatch.rs`) compares the active toolchain channel from `ToolchainDetector` against the MSRV declared in `Cargo.toml` . A mismatch — for example, an `nightly` channel used in a workspace that declares a stable MSRV — yields a "warn" verdict.

TrybuildChangedPolicy (`src/policies/trybuild_changed.rs`) reports whether changed trybuild fixtures were detected. It is informational rather than advisory: the verdict is always "pass" , but the recommendation string reports the count of changed fixtures to guide the practitioner's attention.

3.7.3 MAPE-K Alignment

The Monitor phase is embodied by the adapters, which run unconditionally on every invocation and populate their respective `EngineState` dimensions. The Analyse phase is embodied by the `evaluate()` implementations, each of which reads its relevant dimension(s) and computes a typed result. The Plan phase corresponds to the collection of results into `PolicyState` , from which a unified recommendation surface is constructed. The Execute phase is presently limited to emitting those recommendations as user-visible output and XES evidence events —

no filesystem, git, or network operations are performed without explicit user confirmation.

This constraint is intentional. Autonomic systems that take destructive action without confirmation violate the practitioner's expectation of transparency. The `suggest` -only default is the correct posture for a tool operating in a shared workspace where other processes may be running concurrently.

3.8 Design Decisions and Trade-offs

3.8.1 Single Aggregate Root versus Per-Noun State

The choice to centralise all runtime state into `EngineState` rather than allowing each noun to own its own state carries both benefits and costs. The primary benefit is that policy evaluation always operates on a coherent, simultaneous snapshot of all workspace dimensions. A policy that needs both git state and target size — for example, a hypothetical `BlockPublishPolicy` that refuses to publish from a dirty, oversized workspace — can read both from the same `EngineState` without coordinating between separate state owners. The cost is that populating `EngineState` requires running all adapters, even those whose output is not needed for the current noun. This is mitigated in practice by the speed of the individual adapters (all of which are single subprocess calls or filesystem traversals) and by the `cicd.toml` state cache, which allows adapters to skip expensive re-queries when workspace state has not changed since the last invocation.

3.8.2 External Adjudication via wasm4pm

A defining architectural invariant is that cargo-cicd never adjudicates its own process conformance (Evidence Invariant E1). The XES emission infrastructure exists to produce evidence; the verdict on whether that evidence demonstrates a conformant process is issued exclusively by the external wasm4pm oracle. This separation reflects a fundamental principle: a tool that certifies its own correctness provides no stronger guarantee than an unchecked tool. By routing evidence through an independent oracle, the certification chain has a boundary that cargo-cicd's own test suite cannot corrupt.

The practical consequence is that the release gate is not satisfied by passing internal tests alone. The evidence-gate tests (`tests/wasm4pm_evidence_gate.rs` , `tests/wasm4pm_evidence_mutation.rs` , `tests/wasm4pm_refusal_cases.rs`) must invoke the `wasm4pm` oracle and assert an `Accept` verdict. This is enforced structurally: the `WpmEvidenceOracle` panics if the oracle is unavailable and the expected verdict is not `Blocked` (Evidence Invariant E3).

3.8.3 No Parallel Test Execution

The `--jobs` flag is recognised but not yet functional. Tests are executed serially. This is a deliberate limitation: because `cicd.toml` is a file in the workspace root and all adapters read from and write to the same `target/` directory, concurrent test invocations would risk race conditions on shared state. The correct solution — partitioning state by session identifier — is deferred to a future release. Premature parallelism in a tool that manages workspace state would introduce non-determinism that is difficult to observe and harder to reproduce in CI.

3.8.4 Feature-Flag Isolation of Engine Internals

Compiling the Level 5 engine internals behind the `process-data` feature flag preserves the option of shipping a minimal public binary that has no compile-time dependency on the engine infrastructure. This is valuable for two reasons. First, it enforces the layering discipline at the type level: code that should not reference `EngineState` will fail to compile if it attempts to do so while `process-data` is disabled. Second, it simplifies audit of the public binary surface: reviewers need only inspect the feature-flag-free compilation path to verify that no internal state leaks into public output.

3.8.5 XES as the Evidence Format

The choice of XES (XML Event Stream, as defined by the IEEE XES standard for process mining) as the evidence emission format is driven by the requirements of the `wasm4pm` oracle, which implements conformance checking via token-replay fitness against a process model derived from the declared activity set. JSON or JSONL alone would satisfy a logging requirement, but would not satisfy the conformance checking requirement without additional transformation. The dual emission (XES for oracle adjudication, JSONL for downstream tooling) provides

both the oracle-compatible format and a machine-readable companion for programmatic consumers.

The production XES writer (`emit_xes_filtered`) applies three quality constraints that improve token-replay fitness: (1) only `"complete"` lifecycle events are included, since start events duplicate activity names in the derived Petri net; (2) only the ten declared model activities are included, filtering out noise events such as `"git:status"` that would introduce unmodelled transitions; and (3) events are sorted by timestamp within each trace to ensure the directly-follows graph reflects the actual execution order. These constraints are validated by the mutation and refusal test suites.

3.9 Summary

This chapter has presented the architecture of cargo-cicd as a four-layer system in which a conventional CLI grammar exposes a Level 5 process-data engine. The `EngineState` aggregate root provides a coherent, simultaneous snapshot of all workspace dimensions; the adapter layer translates external sources into typed state without business logic or side effects; the noun-verb CLI grammar scales cleanly across a growing command surface; and `cicd.toml` serves as both configuration carrier and append-only event log. The autonomic policy engine implements the MAPE-K loop in a non-destructive `suggest` -only mode, while the XES evidence infrastructure supports external adjudication by the wasm4pm oracle. The principal design decisions — centralised state, external adjudication, serial test execution, and feature-flag isolation — are conservative choices that prioritise correctness and transparency over performance and flexibility, consistent with the requirements of a tool operating at the boundary of Rust workspace automation and process conformance certification.

Chapter 4: Implementation and Evaluation

4.1 Implementation Overview

cargo-cicd is implemented entirely in Rust and targets a minimum supported Rust version (MSRV) of 1.86, declared explicitly in `Cargo.toml` via the `rust-version` field and enforced at every CI run. The project is structured as a Cargo workspace with three members: the primary `cargo-cicd` binary crate at the workspace root and two supporting library crates, `cargo-cicd-core` and `cargo-cicd-lsp`, under the `crates/` subdirectory.

4.1.1 Dependency Selection

The dependency footprint has been deliberately constrained to a small set of well-audited crates. Table 4.1 summarises the direct runtime dependencies.

Table 4.1 — Runtime Dependencies

Crate	Version	Role
<code>clap</code>	4 (derive)	CLI argument parsing
<code>clap-noun-verb</code>	26.6.2	Noun-verb CLI grammar layer
<code>serde</code>	1 (derive)	Serialisation/deserialisation
<code>toml</code>	0.8	cicd.toml reading and writing
<code>anyhow</code>	1	Error propagation
<code>walkdir</code>	2	Target directory traversal
<code>serde_json</code>	1	JSON output for evidence receipts

Development-time dependencies add `assert_cmd` (2), `tempfile` (3), and `predicates` (3) for integration testing, plus `toml` (0.8) for fixture TOML verification.

`clap-noun-verb` is a local crate developed in tandem with `cargo-cicd`. It implements the noun-verb command grammar that forms the public CLI surface:

	— fixtures/	# FixtureWorkspace helpers
	— invariants.rs	
	— autonomic_policies.rs	
	— wasm4pm_evidence_gate.rs	
	— ...	
	— ontology/cargo-cicd.ttl	# OWL/RDF ontology (manufacturing input)
	— queries/	# SPARQL queries for ggen pipeline
	— templates/	# Tera templates for code generation
	— .github/workflows/	# CI pipeline definitions

4.2 Test Hierarchy

The project enforces a three-tier test hierarchy. Each tier has a distinct scope and a distinct role in the release process.

Table 4.3 — Test Tier Summary

Tier	Files	Tools	Release-blocking?
Tier 1: Unit and smoke	invariants.rs , cli/ , feature_projection.rs , autonomic_policies.rs	assert_cmd , tempfile	No
Tier 2: Integration	cicd_toml_truth.rs , changed_tests.rs , git_phase_closure.rs , ggen_customization_guard.rs	assert_cmd , tempfile , walkdir	No
Tier 3: Evidence gate	wasm4pm_evidence_gate.rs , wasm4pm_evidence_mutation.rs , wasm4pm_refusal_cases.rs	wpm oracle, XES	Yes

Tier 1 tests verify the public CLI boundary, the autonomic policy logic, and the feature flag surface contract. These tests run in every CI job and on both supported platforms. They are fast (typically sub-second), use `tempfile::TempDir` for isolation, and never require an external oracle binary.

Tier 2 tests cover correctness of the cicd.toml carrier format, the changed-test selection algorithm, git phase closure semantics, and the ggen code-generation

customisation guard. These tests exercise multi-step workflows and may spawn subprocesses (`git` , `cargo metadata`).

Tier 3 evidence-gate tests are the release-closing gate. They emit XES (XML Event Stream) evidence files and submit them to the `wasm4pm` oracle (`wpm`) for adjudication. A release may not proceed if the oracle issues a Refuse verdict. This design decouples `cargo-cicd`'s internal correctness claims from external process-compliance verification.

`Cargo.toml` declares each integration test as a separate `[[test]]` entry, ensuring that individual suites can be run in isolation with `cargo test --test <name>` and that their build artefacts are cached independently.

4.3 FixtureWorkspace Testing Strategy

The central abstraction for integration testing is `FixtureWorkspace` , implemented in `tests/fixtures/mod.rs` . Each fixture constructs a minimal but realistic Rust workspace in a `tempfile::TempDir` , populates it with the conditions required to exercise a specific engine verdict, and exposes the workspace root path for use as the `current_dir` of `assert_cmd` invocations. The `TempDir` is owned by the `FixtureWorkspace` struct; when the struct is dropped at the end of a test, the operating system reclaims the temporary directory automatically, providing strong isolation between tests.

4.3.1 Available Fixtures

Table 4.4 — FixtureWorkspace Variants

Constructor	Preconditions	Expected Verdict
<code>FixtureWorkspace::clean()</code>	Valid <code>Cargo.toml</code> , fully committed, no <code>target/</code> , no <code>cicd.toml</code>	Pass
<code>FixtureWorkspace::dirty()</code>		Warn (git dirty)

Constructor	Preconditions	Expected Verdict
	Clean baseline + one untracked file (<code>untracked.txt</code>)	
<code>FixtureWorkspace::missing_manifest()</code>	Empty directory, no <code>Cargo.toml</code>	Refuse
<code>FixtureWorkspace::with_toolchain_mismatch()</code>	Clean + <code>rust-toolchain.toml</code> pinning channel <code>1.50.0</code>	Warn
<code>FixtureWorkspace::with_target_over_limit()</code>	Clean + <code>target/debug/placeholder.bin</code> (1,048,576 bytes)	Warn (target pressure)
<code>FixtureWorkspace::with_corrupted_cicd_toml()</code>	Clean + syntactically invalid <code>cicd.toml</code>	Fail/Refuse
<code>FixtureWorkspace::with_stale_cicd_toml()</code>	Clean + <code>cicd.toml</code> claiming <code>dirty = false</code> , then made dirty	Warn (cache mismatch)
<code>FixtureWorkspace::with_changed_trybuild_fixture()</code>	Clean + <code>tests/ui/</code> containing 10 unchanged + 1 changed fixture	Pass (changed-only)

4.3.2 Isolation Guarantees

Each fixture satisfies four isolation properties:

1. **Filesystem isolation.** The `TempDir` is created in the system temporary directory, never inside the repository. Tests therefore never modify the working checkout.
2. **Git isolation.** Fixtures that require git history initialise a fresh git repository via `git init` within the `TempDir` . The global git configuration of the host is not modified.

3. **State isolation.** Fixtures that place a `cicd.toml` in the workspace do so programmatically, with exactly the state required by the test scenario. There is no shared mutable state between tests.
4. **Drop isolation.** Because `TempDir` implements `Drop`, cleanup occurs even when a test panics. The cargo test harness runs each `#[test]` function in its own thread; a panic in one thread does not prevent other tests from running.

4.3.3 Example: Verifying a Dirty Workspace

The following pattern, drawn from the project's own test suite, illustrates the canonical fixture usage:

```
#[test]
fn test_dirty_workspace_verdict() {
    let fixture = FixtureWorkspace::dirty();
    let output = Command::cargo_bin("cargo-cicd")
        .unwrap()
        .arg("status")
        .current_dir(&fixture.root)
        .output()
        .unwrap();
    assert!(String::from_utf8_lossy(&output.stdout).contains("dirty"));
}
```

The `fixture` variable owns the `TempDir`. The `Command::cargo_bin` call resolves the built binary from `CARGO_BIN_EXE_cargo-cicd`, eliminating reliance on the system `PATH`.

4.4 Invariant Testing

`tests/invariants.rs` encodes the non-negotiable public boundary invariants. These invariants are enforced on every CI push and pull request across all platform and toolchain combinations. The file currently implements four invariant functions, each named with the `invariant_` prefix to make the suite's purpose self-documenting.

4.4.1 Invariant 1: No Forbidden Terms in Public Output

The most comprehensive invariant iterates over all public CLI entry points (top-level `--help` and per-noun `--help`) and asserts that none of the following terms appear in stdout or stderr:

```
ALIVE, Nehemiah, CONSTRUCT8, Instinct8, Inspection Gate,  
Cargo Court, AGI, Truex, Field8, wall
```

These terms belong to the internal architecture and must never leak into user-visible output. The test runs `cargo-cicd` with nine distinct argument lists and checks both output streams:

```
let text = String::from_utf8_lossy(&output.stdout).to_string()  
    + &String::from_utf8_lossy(&output.stderr);  
for term in &forbidden {  
    assert!(  
        !text.contains(term),  
        "Forbidden term '{term}' found in output of: cargo cicd {args}",  
        term, args.join(" ")  
    );  
}
```

A parallel check in `tests/ggen_customization_guard.rs` extends this verification to the file system: it walks `README.md` and all Markdown files under `docs/tutorials/`, `docs/how-to/`, `docs/reference/`, and `docs/explanation/`, asserting the same forbidden-term list. This dual enforcement (CLI output and static documents) closes the gap between runtime behaviour and shipped documentation.

4.4.2 Invariant 4: No Destructive Default

`invariant_no_destructive_default_target_prune_is_safe` constructs a temporary directory containing a `target/debug/` tree with a synthetic binary, runs `cargo cicd target prune` without any confirmation flag, and then asserts that the binary still exists:

```
assert!(  
    fake_target.join("binary").exists(),
```

```
"target prune without --confirm must not delete files"
);
```

This invariant directly encodes the design constraint that no cargo-cicd command may take a destructive action without explicit user confirmation. The `--confirm` flag is required to activate deletion.

4.4.3 Invariant 5: No Full Trybuild by Default

The trybuild invariant creates 100 synthetic `tests/ui/compile_fail/fixture_N.rs` files in a temporary workspace and invokes `cargo cicd trybuild changed`. The assertion is negative: the combined output must not contain the strings `"100 fixtures"` or `"all 100"`. The invariant enforces that the changed-only selection algorithm is never bypassed, regardless of the number of fixtures present.

4.4.4 Invariant 6: wasm4pm Scan or Documented Absence

The sixth invariant enforces a process-compliance property rather than a code-correctness property. It checks whether at least one of three paths exists: the wasm4pm capability scan receipt, the integration recommendation document, or the deferred-work document. If none exists, the test logs a `PARTIAL` message but does not fail, because the scan workflow may be running concurrently. The invariant is about the process — that capability assessment was performed and its outcome was recorded — not about the binary timing of that recording.

4.4.5 ggen Customisation Guard

`tests/ggen_customization_guard.rs` protects the code-generation manufacturing pipeline. It verifies that every `BEGIN ggen:` marker in `README.md` has a matching `END ggen:` marker (and vice versa for `BEGIN custom:`), that the README contains a command table generated from the ontology, and that reference documentation files exist for every public command. The `evidence_emission_not_removed` test specifically guards against accidental removal of the `ProcessEvent` struct or the `emit_xes` function from `src/evidence.rs`.

Table 4.5 — Invariant Tests at a Glance

Test name	What it enforces
<code>invariant_public_boundary_no_forbidden_terms_in_all_help</code>	10 forbidden terms absent from all 9 public help surfaces
<code>invariant_no_false_close_git_close_help_mentions_safety</code>	<code>git close --help</code> acknowledges safety (informational)
<code>invariant_no_destructive_default_target_prune_is_safe</code>	Files survive <code>target prune</code> without <code>--confirm</code>
<code>invariant_no_full_trybuild_by_default</code>	100-fixture workspace does not trigger full run
<code>invariant_wasm4pm_scan_or_documented_absence</code>	wasm4pm capability was assessed and outcome recorded

4.5 wasm4pm Evidence Gate

The wasm4pm evidence gate is the release-closing mechanism for cargo-cicd. It implements a strict separation between internal test assertions (which verify cargo-cicd's own behaviour) and process-compliance verification (which is delegated entirely to the external `wpm` oracle).

4.5.1 Evidence Format and Emission

Process evidence is emitted as XES (XML Event Stream), the standard format specified by the IEEE Process Mining standard. Each invocation of a public verb emits one or more `ProcessEvent` records. The `emit_xes` function in `src/evidence.rs` serialises these records to an XES file at `target/cargo-cicd/evidence/` within the workspace. The XES format was chosen over JSONL or custom CSV because it is natively understood by `wpm`; no translation layer is required.

A minimal evidence emission sequence:

```
let events = vec![ProcessEvent::new("status show", "PASS")];
let xes_path = dir.path().join("events.xes");
emit_xes(&events, &xes_path).expect("emit_xes must not fail");
assert!(xes_path.exists(), "XES file must exist before oracle call");
```

4.5.2 Oracle Protocol

The `wpmEvidenceOracle` struct encapsulates binary discovery and invocation. At runtime, it resolves the `wpm` binary through three resolution stages: an explicit `WPM_PATH` environment variable, the known installation path `/Users/sac/wasm4pm/target/release/wpm`, and finally the system `PATH` via which `wpm`. This resolution order ensures that CI environments with a centrally installed `wpm` are preferred over developer-local builds.

Once the oracle is resolved, the primary audit command is:

```
wpm audit <file.xes>
```

The secondary receipt doctor command is:

```
wpm receipt doctor --format json --strict <receipt.json>
```

Both commands must return a non-zero-free exit status and must not produce `FAIL` or `REFUSE` in their combined output for the evidence gate to pass.

4.5.3 Oracle-Absent Fallback

When the `wpm` binary is absent (as in standard CI runners that do not have `wasm4pm` installed), each evidence-gate test falls back to an `ExpectedWpmVerdict::Blocked` assertion. This allows the test suite to complete without the oracle while making it transparent that the Accept branch was not exercised. For release closure, pipeline operators set `REQUIRE_WPM_ORACLE=1` ; under that environment variable, binary absence causes an immediate panic rather than a silent skip:

```
fn absent_oracle_verdict(test_name: &str) -> ExpectedWpmVerdict {
    if std::env::var("REQUIRE_WPM_ORACLE").as_deref() == Ok("1") {
        panic!(
            "REQUIRE_WPM_ORACLE=1 is set but the wpm oracle binary is absent. \
            Test '{} cannot exercise its Accept assertion.",
            test_name
        );
    }
    ExpectedWpmVerdict::Blocked
}
```

4.5.4 Hard Gate Test

`evidence_gate_wpm_doctor_hard_gate` is a mandatory test that invokes `wpm doctor` directly and asserts both exit code zero and the absence of failure indicators in the combined output. Unlike the per-verb acceptance tests, this test targets the oracle's self-diagnosis facility. If `wpm doctor` reports any internal failure, the release is blocked regardless of the per-verb results.

Table 4.6 — Evidence Gate Tests

Test	Verb under test	Evidence submitted
<code>evidence_gate_status_show_accepted</code>	<code>status show</code>	Single PASS event
<code>evidence_gate_target_show_accepted</code>	<code>target show</code>	Single PASS event
<code>evidence_gate_target_prune_accepted</code>		

Test	Verb under test	Evidence submitted
	target prune	Single DRY-RUN event
evidence_gate_changed_test_accepted	test changed	Single PASS event
evidence_gate_git_close_accepted	git close	Single PASS event
evidence_gate_publish_run_accepted	publish run	Single PASS event
evidence_gate_workspace_doctor_accepted	workspace doctor	Single PASS event
evidence_gate_oracle_discover	—	Oracle self-probe (no panic)
evidence_gate_wpm_doctor_hard_gate	—	wpm doctor self-diagnosis

4.6 CI/CD Pipeline

The GitHub Actions pipeline is defined in `.github/workflows/ci.yml` and consists of five jobs that run in parallel after checkout.

4.6.1 Pipeline Jobs

fmt-clippy runs on a 2×1 matrix (`ubuntu-latest` , `macos-latest`). It installs the stable Rust toolchain with the `rustfmt` and `clippy` components and runs `cargo fmt --all -- --check` followed by `cargo clippy --all-targets -- -D warnings` in two passes: once with default features and once with `--all-features` . Clippy warnings are treated as errors.

test runs on a 2×2 matrix (2 platforms × 2 toolchains), producing four concurrent runners.

Table 4.7 — Test Matrix

	ubuntu-latest	macos-latest
stable	ubuntu/stable	macos/stable
1.86 (MSRV)	ubuntu/1.86	macos/1.86

Each runner in the test matrix executes the following sequence: 1.

`cargo build --workspace` — verifies the full workspace compiles
2. Individual test suites: `invariants`, `cli`, `cicd_toml_truth`, `autonomic_policies`, `changed_tests`, `git_phase_closure`, `feature_projection`
3. `cargo test --workspace` — runs all tests with default features

Test results are uploaded as artefacts with a 14-day retention period, keyed by platform and toolchain combination.

feature-matrix runs on `ubuntu-latest` against four feature combinations: "" (default), `process-data`, `autonomic`, and `wasm4pm`. Both `cargo build` and `cargo test` are executed for each combination, ensuring that no feature flag introduces a compilation failure or test regression.

forbidden-terms is a static analysis job that runs on `ubuntu-latest`. It uses `grep` to scan `src/**/*.rs` for forbidden terms on non-comment lines, scans CLI help strings specifically, and scans public Markdown files (`README.md`, `docs/reference/`, `docs/agents/`). Internal documentation directories (`docs/wasm4pm/`, `docs/release/`, `docs/design/`, `receipts/`) are excluded from the scan.

workspace-integrity verifies that `Cargo.lock` is consistent with `Cargo.toml`, that all workspace members resolve via `cargo metadata`, and that the `rust-version` field is present and equal to `1.86`. The MSRV check is implemented as a Python one-liner that parses the JSON output of `cargo metadata`:

```
- name: Verify MSRV is declared and matches rust-version field
  run: |
    MSRV=$(cargo metadata --format-version 1 --no-deps \
```

```
| python3 -c "import sys,json; m=json.load(sys.stdin); ...")
if [ "$MSRV" != "1.86" ]; then
    echo "::error::MSRV mismatch – expected 1.86, got $MSRV"
    exit 1
fi
```

4.7 Performance Characteristics

cargo-cicd is designed to complete a full `cargo cicd status` invocation in under one second on a typical developer workstation. Three design decisions contribute to this target.

4.7.1 Single-Invocation Git Queries

All git state is captured with a single `git status --porcelain` invocation. The output is parsed once and cached in `GitPhaseState` for the duration of the session. Multiple adapters (dirty flag, untracked count, changed file list) all read from this single cached parse. The alternative — issuing separate `git ls-files`, `git diff`, and `git status` calls — would multiply subprocess overhead and introduce races on systems with high I/O latency.

4.7.2 Bounded Target Directory Traversal

The `TargetScannerAdapter` uses `walkdir` with a `max_depth(3)` limit, avoiding deep traversal of nested build artefacts. The adapter accumulates only the total byte count, not a per-file inventory, which keeps memory usage constant regardless of workspace size. The computed size is persisted in the `[state]` section of `cicd.toml` and is only recomputed when the HEAD commit hash has changed since the last run.

4.7.3 cicd.toml State Cache

`cicd.toml` functions as an inter-invocation state cache. On each run, adapters consult the cached value first and re-query the external source only when the cached value may be stale (determined by comparing the recorded HEAD hash to the current HEAD). For expensive operations such as parsing `Cargo.lock` for

dependency tree analysis, this caching pattern reduces steady-state cost to a single file read plus a hash comparison.

Table 4.8 — Performance Design Choices

Operation	Naive approach	Chosen approach	Benefit
Git state	Multiple <code>git ls-files</code> + <code>git diff</code>	One <code>git status --porcelain</code>	Fewer subprocesses
Target size	Full recursive walk	<code>walkdir</code> with <code>max_depth(3)</code> , byte sum only	Bounded traversal
Cargo metadata	Per-crate <code>cargo metadata</code>	Single <code>cargo metadata --format-version 1</code>	Linear not quadratic
Cross-invocation state	Re-query everything	<code>cicd.toml</code> cache keyed by HEAD hash	Sub-second repeat runs

4.8 Evaluation Results

4.8.1 Test Suite Scale

The project declares 16 integration test binaries in `Cargo.toml`. The named suites cover the following domains:

Table 4.9 — Named Integration Test Suites

Suite	Domain
<code>feature_projection</code>	Feature flag surface contract
<code>cli</code>	CLI command projection
<code>cicd_toml_truth</code>	<code>cicd.toml</code> schema and write correctness
<code>autonomic_policies</code>	Autonomic policy verdicts
<code>changed_tests</code>	Changed-test selection algorithm
<code>git_phase_closure</code>	Git phase state transitions

Suite	Domain
<code>invariants</code>	Non-negotiable public boundary invariants
<code>wasm4pm_harness</code>	Evidence harness smoke tests
<code>wasm4pm_evidence_gate</code>	Positive acceptance evidence gate
<code>wasm4pm_evidence_mutation</code>	Mutation-adversarial evidence gate
<code>wasm4pm_refusal_cases</code>	Refuse-path evidence gate
<code>ggen_customization_guard</code>	Code-generation guard
<code>refusal_calibration</code>	Calibration of refusal thresholds
<code>lsp_explain</code>	LSP explain surface
<code>fixture_workspaces</code>	FixtureWorkspace construct correctness
(implicit) <code>interactions</code>	Cross-noun interaction coverage

4.8.2 Policy Evaluation Coverage

The `autonomic_policies` suite tests four policies across multiple input regions:

Table 4.10 — Autonomic Policy Test Coverage

Policy	Pass condition	Warn condition	Suggest condition
<code>target_pressure</code>	size < 80% of limit	80%–100% of limit	> limit
<code>toolchain_mismatch</code>	no pinned channel, or channels match	—	channels differ
<code>trybuild_changed</code>	0 changed fixtures	—	≥ 1 changed fixture
<code>git_phase_dirty</code>	0 dirty files	—	≥ 1 dirty file

A cross-cutting invariant asserts that every policy defaults to `PolicyMode::Suggest`, regardless of verdict. No policy may activate in `Apply` mode without explicit user opt-in. This invariant is encoded as a parametric test:

```
#[test]
fn test_no_policy_uses_apply_mode_by_default() {
```

```

    for r in &[
        check_target_pressure(5.0, 20.0),
        check_toolchain_mismatch("stable", None),
        check_trybuild_changed(0),
        check_git_phase_dirty(0),
    ] {
        assert!(matches!(r.mode, PolicyMode::Suggest), ...);
    }
}

```

A second cross-cutting invariant asserts that all policies are enabled by default and that every `Pass` verdict yields an empty recommendation string.

4.8.3 Platform Coverage

The 2×2 test matrix produces four configurations on every push to `main` and every pull request. The MSRV configuration (1.86) ensures that no dependency silently raises the minimum compiler requirement. The stable configuration tracks the current release of the Rust toolchain and catches regressions introduced by new Clippy lints or standard library changes.

4.9 Known Limitations

4.9.1 Feature Flag Gating

When built without the `process-data` feature (the default), the Level 5 engine internals are entirely absent from the binary. `EngineState`, all adapters, `cicd.toml` I/O, policy evaluation, and XES emission are compiled out. This is intentional: the public CLI surface functions correctly in the default configuration, and internal plumbing is opt-in. However, it means that a user installing `cargo-cicd` from `crates.io` without specifying features receives a CLI that does not emit process evidence or evaluate autonomic policies.

4.9.2 Policy Apply Mode Not Implemented

The `--apply` flag is recognised in the CLI grammar but is not yet functional. All policies operate exclusively in `suggest` mode. Automated remediation — such as

running `cargo clean` when `TargetPressurePolicy` fires — is deferred pending additional field testing of the policy calibration thresholds.

4.9.3 Serial Test Execution

`cargo-cicd` runs tests serially. The `--jobs` flag is parsed but ignored. The reason is architectural: because `cicd.toml` is a global file in the workspace root, concurrent test processes modifying it would create a write race. Addressing this limitation would require either per-test `cicd.toml` namespacing or a locking protocol, both of which are deferred to a post-v26.6.2 release.

4.9.4 Git Requirement

All state-tracking features require the workspace to be a git repository. Non-git workspaces receive a degraded experience: git-dependent adapters return empty state, and any test that spawns `git status` will fail. The integration test suite handles this by always initialising a fresh git repository inside the `TempDir`.

4.9.5 Platform Coverage

The CI matrix covers Linux (`ubuntu-latest`) and macOS (`macos-latest`). Windows is not tested. Path separator differences and the behaviour of `git status --porcelain` on Windows (particularly with respect to line endings) have not been validated. Users on Windows may encounter path-construction errors in the adapter layer.

4.9.6 wasm4pm Oracle Availability

The evidence-gate tests exercise the Accept branch only when the `wpm` binary is present. In standard GitHub Actions runners the binary is absent, so the Accept branch is silently skipped unless `REQUIRE_WPM_ORACLE=1` is set. Release engineers are responsible for configuring a self-hosted runner with `wpm` installed, or for running the evidence-gate suite locally against a known-good oracle installation before tagging a release.

4.9.7 No Workspace Federation

`cargo-cicd` recognises exactly one workspace root per invocation, determined by the current working directory or an explicit `--root` flag. Monorepos that contain multiple Cargo workspaces at different directory levels are not supported. Cross-

workspace test dependencies are not modelled in `TestPlanState` , and the `cicd.toml` carrier does not support workspace federation semantics.

Chapter 5: Conclusion and Future Work

5.1 Summary of Contributions

This dissertation set out to answer three research questions that motivated the design and implementation of cargo-cicd:

RQ1: Can a structured process-data engine be embedded within a conventional developer CLI tool without exposing internal machinery to end users?

RQ2: Does a noun-verb command grammar, derived from an ontological manufacturing pipeline, produce a more consistent and extensible DevOps CLI surface than ad-hoc subcommand hierarchies?

RQ3: Can local-first CI/CD orchestration — running entirely on the developer's machine, prior to any remote pipeline invocation — reduce integration failures and build cache pressure in Rust workspaces at a rate comparable to dedicated cloud CI systems?

The thesis answers all three questions affirmatively. cargo-cicd demonstrates that a Level 5 process-data engine — one whose internal state, evidence emission, and policy evaluation are architecturally distinct from its public interface — can be manufactured and deployed as an ordinary Cargo subcommand. End users interact with a simple noun-verb CLI (`cargo cicd status` , `cargo cicd test changed`) while the engine silently accumulates workspace state across eleven structured dimensions, emits XES-formatted process evidence, and optionally submits that evidence to an external adjudicator (the wasm4pm oracle) for independent verdict.

The key achievement is the clean layering: the public boundary presents no forbidden terms, no engine internals, and no process-mining vocabulary. The private boundary, gated behind the `process-data` feature flag, exposes the full Level 5 engine to contributors, integration tests, and the release-gate evidence pipeline. This separation was enforced not by convention but by the Rust feature flag system and validated at every commit by the `invariants` integration test suite.

5.2 Theoretical Contributions

5.2.1 The Level 5 Engine Taxonomy

Prior work in DevOps tooling has characterised CI/CD systems at four levels of maturity: scripted pipelines (Level 1), parameterised pipelines (Level 2), reusable pipeline libraries (Level 3), and policy-driven pipelines (Level 4) [1, 2]. This dissertation proposes Level 5 as a distinct category: a *process-data engine* that treats every developer action as a structured event in a process trace, models workspace state as an aggregate root, and enables post-hoc adjudication of process conformance by an external oracle.

The distinction between Level 4 and Level 5 is not merely one of sophistication. Level 4 systems enforce policy at invocation time — a rule fires, a build is blocked. Level 5 systems accumulate evidence of *process intent* alongside outcomes, enabling retrospective conformance analysis, cross-run comparison, and formal acceptance verdicts. This is the core theoretical novelty of cargo-cicd: the tool does not merely enforce rules; it manufactures evidence that rules were followed.

5.2.2 Adapter Pattern Formalisation for DevOps State

The adapter pipeline introduced in Chapter 3 formalises a pattern that is common in practice but rarely stated precisely. Each adapter in cargo-cicd satisfies three invariants: (i) it reads exactly one external source, (ii) it performs no business logic — only translation — and (iii) it is stateless across invocations. This triple invariant ensures that `EngineState` is the single source of truth and that adapter correctness can be tested in isolation using fixture workspaces.

This formalisation extends the classical Adapter pattern [3] by adding the *single-source* constraint, which is non-trivial in DevOps contexts where external sources (git, cargo, rustup, the filesystem) frequently overlap in the information they expose. For example, both `git status --porcelain` and `cargo metadata` can reveal the presence of uncommitted changes to `Cargo.lock`; the single-source invariant requires one adapter to own that fact, eliminating the risk of contradictory state.

5.2.3 Noun-Verb Grammar for DevOps CLIs

Chapter 2 introduced the `clap-noun-verb` grammar, derived from an OWL ontology (`ontology/cargo-cicd.ttl`) via SPARQL queries and Tera templates. The grammar encodes DevOps commands as *noun* (the resource: `target` , `test` , `git` , `workspace`) and *verb* (the action: `show` , `prune` , `changed` , `close`). Default-verb injection allows bare nouns to work without subcommand disambiguation, preserving ergonomics while enforcing structural consistency.

The contribution is twofold. First, the grammar is *manufactured*: nouns and verbs are generated from the ontology, not written by hand, which means the CLI surface and its formal specification remain in sync. Second, the grammar is *extensible*: adding a noun requires registering it in the ontology, regenerating, and implementing the generated scaffolding — a process that enforces interface consistency without requiring contributors to audit existing code. This approach builds on prior work in domain-specific language design [4, 5] and ontology-driven code generation [6], applying those ideas specifically to the DevOps CLI domain.

5.3 Practical Contributions

5.3.1 The cargo-cicd Tool

`cargo-cicd` v26.6.2 is a production-quality Cargo subcommand distributed on crates.io under the MIT/Apache-2.0 dual license. It provides nine commands across seven nouns, covering the complete local CI/CD lifecycle: workspace health diagnosis, git state inspection, changed-test selection, trybuild fixture management, target directory maintenance, and crate publishing. The tool requires no network access for its core operations, runs in under two seconds on representative workspaces, and integrates directly into GitHub Actions pipelines.

5.3.2 The cicd.toml Schema

The `cicd.toml` file is a local state carrier whose schema is defined in `src/cicd_toml.rs` . It persists cross-run state (last-known target size, last HEAD hash, last test verdicts) and configuration (`[target]` , `[test.changed]` , `[autonomic]`), enabling subsequent invocations to skip expensive recomputation. The schema is versioned and validated at load time; a corrupted or stale `cicd.toml` is treated as a `refuse` verdict by the `WorkspaceDoctorEvent` , not

as a silent error. The schema design draws on prior work in local configuration file standards [7] and workspace state management [8].

5.3.3 The wasm4pm Integration Protocol

The wasm4pm integration protocol, formalised in Chapter 4, defines the contract between cargo-cicd (the evidence emitter) and the wasm4pm oracle (the adjudicator). Evidence is emitted in XES format [9] to `target/cargo-cicd/evidence/`, with one XES file per invocation. The `wpm receipt doctor --format json --strict` command validates structured receipts; `wpm audit <file.xes>` validates raw XES traces. Release closure requires both commands to return an `Accept` verdict. This protocol instantiates the broader process-mining conformance checking paradigm [10, 11] in a local-first DevOps context, demonstrating that formal process conformance need not require a cloud infrastructure or a process-mining server.

5.4 Lessons Learned

5.4.1 Rust for DevOps Tooling

Rust proved well-suited to DevOps tooling for reasons beyond performance. The ownership model naturally enforces the single-source invariant: only one adapter can hold a mutable reference to a state dimension at a time, making race conditions in the adapter pipeline a compile-time error rather than a test-time discovery. The `cfg(feature = ...)` attribute provided a clean mechanism for the two-tier public/private boundary, enforced by the type system at every call site. The `anyhow` crate's error propagation idioms kept adapter code concise without sacrificing error context.

The primary challenge was the Minimum Supported Rust Version (MSRV) constraint. cargo-cicd requires Rust 1.85 or later, driven by dependencies that use stabilised features from the 2024 edition. Managing MSRV in a tool that targets developers running older stable channels required explicit `rust-version` declarations in `Cargo.toml` and a `rust-toolchain.toml` fixture in the test suite. Future tooling in this space should establish MSRV policy before selecting dependencies, not after.

5.4.2 Test Isolation via Fixture Workspaces

The `FixtureWorkspace` pattern — an ephemeral git repository created in a `TempDir` for each integration test, with a known initial state — proved essential for test reliability. Without fixture isolation, integration tests would depend on the developer's local git state, Cargo cache, and toolchain configuration, making them flaky on CI and impossible to reproduce. The eight fixture archetypes (`clean` , `dirty` , `missing_manifest` , `with_toolchain_mismatch` , `with_target_over_limit` , `with_corrupted_cicd_toml` , `with_stale_cicd_toml` , `with_changed_trybuild_fixture`) cover the seven invariant verdicts and provide a vocabulary for describing workspace health states.

A key lesson is that fixture workspaces must be valid git repositories from their first commit. Several early test failures were caused by adapters that invoked `git status --porcelain` in a directory that had never been committed to, producing unexpected output. The solution — initialising and committing the fixture in its constructor — is now encoded in the `FixtureWorkspace::clean()` implementation and serves as the base for all derived fixtures.

5.4.3 The Cost of Deferred Observability

cargo-cicd v26.6.2 does not use a standard logging framework. Instrumentation is via `ProcessEvent` emission to XES and direct `println!` / `eprintln!` calls. This was a deliberate early decision to avoid the `tracing / env_logger` dependency weight while the architecture was still evolving. In retrospect, the absence of structured logging added significant friction during adapter debugging: reproducing a failure required adding temporary `eprintln!` statements, rebuilding, and re-running the test. A structured logging framework, initialised early, would have reduced this friction substantially without adding meaningful binary size for release builds (which strip debug symbols).

5.5 Future Work

The following directions are identified as high-priority extensions for the cargo-cicd research programme. They are ordered from most architecturally constrained (requiring core changes) to most independent (additive features).

5.5.1 Remote State Synchronisation for `cicd.toml`

The current `cicd.toml` is intentionally local-only: it is added to `.gitignore` and is not pushed to remote. This means that a developer switching machines, or a CI system cloning the repository fresh, starts with no cached state and incurs the full cost of cold-start workspace analysis on every run. Remote state synchronisation would allow `cicd.toml` to be stored in a lightweight remote store (a git-notes ref, a repository-scoped artifact store, or a dedicated key-value backend) and fetched on clone. The primary challenge is conflict resolution: two developers modifying the same workspace concurrently may produce divergent `cicd.toml` states. A merge strategy based on last-write-wins per state dimension, with conflict detection for configuration keys, is the most promising approach.

5.5.2 Workspace Federation for Monorepos

cargo-cicd currently recognises a single workspace root, identified by the presence of a `[workspace]` section in `Cargo.toml`. Monorepos that span multiple Cargo workspaces — a common pattern in organisations that maintain both a core library workspace and a tools workspace in the same repository — are not supported. Workspace federation would allow a single `cicd.toml` at the repository root to aggregate state from multiple workspace roots, with per-workspace configuration sections and a unified policy evaluation pass. This requires extending the `EngineState` aggregate to carry a vector of `WorkspaceState` instances and modifying all adapters to accept a workspace selector.

5.5.3 Policy Apply Mode

The autonomic policy engine currently operates in `suggest` mode: policies evaluate workspace state and emit recommendations, but no action is taken without explicit user intervention. The `--apply` flag is parsed but not implemented. Full policy apply mode would allow policies to take direct action — pruning stale build artifacts, running the formatter, staging changed fixture files — subject to a confirmation prompt and an audit trail in the XES evidence log. The primary safety requirement is idempotency: a policy applied twice must produce the same result as applied once. This is already satisfied for `TargetPressurePolicy` (pruning is idempotent) but requires careful specification for `GitPhaseDirtyPolicy` (which may need to stage and commit files).

5.5.4 Parallel Test Execution with State Locking

The current test runner serialises test execution to avoid concurrent writes to `cicd.toml` and the XES evidence directory. For large workspaces with many independent crates, this serialisation is the dominant source of latency. Parallel test execution would run tests for independent crates concurrently, with a file-level lock on `cicd.toml` and a per-session evidence directory (using the session ID as a path component). The `--jobs N` flag is already recognised by the CLI; the implementation requires a work-stealing scheduler and a lock-free merge of per-job `TestPlanState` results into the aggregate `EngineState`.

5.5.5 Custom Policy Plugin System

Built-in policies cover the invariant workspace health conditions: git dirty state, target directory pressure, toolchain mismatch, and manifest validity. Organisation-specific policies — enforcing internal dependency version alignment, checking for approved license identifiers, or requiring specific commit message formats — cannot currently be expressed. A plugin system would allow policies to be defined as dynamic libraries (`.so` / `.dylib` / `.dll`) or as WebAssembly modules, loaded at runtime from a `[plugins]` section in `cicd.toml`. The WebAssembly approach is architecturally preferred: it is sandboxed, cross-platform, and composable with the existing wasm4pm integration.

5.5.6 Windows Cross-Compilation Support

cargo-cicd is tested on Linux and macOS. Windows support is blocked by two issues: path separator handling in `TargetScannerAdapter` (which uses Unix-style path separators in several string operations) and the absence of a Windows CI runner in the release pipeline. Resolving these issues would expand the addressable developer population significantly, as a substantial fraction of Rust developers work on Windows. The changes are largely mechanical — replacing string-based path operations with `std::path::PathBuf` throughout — but require a Windows test runner and fixture workspace validation on NTFS.

5.5.7 Tracing and Observability Integration

Structured logging via the `tracing` crate [12] would provide three benefits: (i) runtime-configurable verbosity via `RUST_LOG`, eliminating the need for recompilation during debugging; (ii) span-level timing for adapter calls, enabling

performance regression detection; and (iii) integration with `tracing-opentelemetry` for export to observability backends. The integration path is well-defined: add `tracing` and `tracing-subscriber` to `[dependencies]`, initialise a subscriber in `main()`, and replace `eprintln!` calls in adapters with `tracing::debug!` and `tracing::info!`. The XES evidence pipeline is orthogonal to the tracing infrastructure and would not be replaced; tracing serves developer debugging, while XES evidence serves process conformance.

5.6 Closing Remarks

`cargo-cicd` began as a response to a practical observation: Rust workspace developers routinely push code that fails CI for reasons that are locally detectable — dirty git trees, bloated `target/` directories, broken `trybuild` fixtures, stale toolchain pins. A tool that checks these conditions in two seconds, before any network round-trip, eliminates an entire class of CI failures and recovers developer time that would otherwise be spent waiting for remote pipelines.

The contribution of this dissertation is to show that such a tool need not be a collection of shell scripts. By modelling the workspace as an aggregate of structured state dimensions, populating that state through a pipeline of single-source adapters, and emitting every action as process evidence in a standard format, `cargo-cicd` becomes something more than a linter: it becomes a process-data engine whose behaviour can be formally verified by an external oracle. The `wasm4pm` integration protocol demonstrates that local-first CI/CD and formal process conformance are not in tension — they are complementary.

The noun-verb grammar, manufactured from an ontological specification, enforces a discipline that CLI tools rarely achieve: the public interface is a formal projection of a formal model, not an accretion of ad-hoc subcommands. This discipline pays dividends as the tool grows: adding a noun to the ontology propagates a consistent interface through code generation, documentation, and test scaffolding without requiring contributors to audit or imitate existing code.

The limitations documented in Section 5.5 are real constraints, not theoretical gaps. Remote state synchronisation, workspace federation, and the custom policy plugin system each require non-trivial architectural work. But the foundation laid by `cargo-cicd` v26.6.2 — the `EngineState` aggregate, the adapter pipeline, the feature-

gated two-tier boundary, the wasm4pm evidence protocol — is designed to support these extensions without requiring a rewrite. The Level 5 engine taxonomy is not a ceiling; it is a floor.

References

- [1] Humble, J. and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
- [2] Forsgren, N., Humble, J. and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- [3] Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [4] Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley Professional.
- [5] Mernik, M., Heering, J. and Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4), pp. 316–344.
- [6] Volter, M. and Stahl, T. (2006). *Model-Driven Software Development: Technology, Engineering, Management*. John Wiley & Sons.
- [7] Preston-Werner, T. (2013). TOML: Tom's Obvious, Minimal Language. GitHub repository. <https://github.com/toml-lang/toml>
- [8] Beller, M., Gousios, G. and Zaidman, A. (2017). Oops, my tests broke the build: an explorative analysis of Travis CI with GitHub. In *Proceedings of the 14th International Conference on Mining Software Repositories (MSR)*, pp. 356–367. IEEE.
- [9] Verbeek, H. M. W., Buijs, J. C. A. M., van Dongen, B. F. and van der Aalst, W. M. P. (2010). XES, XESame, and ProM 6. In *Proceedings of the CAiSE Forum*, Lecture Notes in Business Information Processing, vol. 72, pp. 60–75. Springer.

- [10] van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action*. 2nd ed. Springer.
- [11] Carmona, J., van Dongen, B., Solti, A. and Weidlich, M. (2018). *Conformance Checking: Relating Processes and Models*. Springer.
- [12] Tokio Contributors (2019). tracing: Application-level tracing for Rust. <https://docs.rs/tracing>
- [13] Matsakis, N. D. and Klock, F. S. (2014). The Rust programming language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT)*, pp. 103–104. ACM.
- [14] Jung, R., Jourdan, J.-H., Krebbers, R. and Dreyer, D. (2021). Safe systems programming in Rust. *Communications of the ACM*, 64(4), pp. 144–152.
- [15] Klabnik, S. and Nichols, C. (2019). *The Rust Programming Language*. No Starch Press.
- [16] van der Aalst, W. M. P., Weijters, A. J. M. M. and Maruster, L. (2004). Workflow mining: discovering process models from event logs. *IEEE Transactions on Knowledge and Data Engineering*, 16(9), pp. 1128–1142.
- [17] Leemans, S. J. J., Fahland, D. and van der Aalst, W. M. P. (2013). Discovering block-structured process models from event logs — a constructive approach. In *Proceedings of the 4th International Conference on Application and Theory of Petri Nets and Concurrency (PETRI NETS)*, Lecture Notes in Computer Science, vol. 7927, pp. 311–329. Springer.
- [18] Kephart, J. O. and Chess, D. M. (2003). The vision of autonomic computing. *IEEE Computer*, 36(1), pp. 41–50.
- [19] Huebscher, M. C. and McCann, J. A. (2008). A survey of autonomic computing — degrees, models, and applications. *ACM Computing Surveys*, 40(3), pp. 1–28.
- [20] McIlroy, M. D., Pinson, E. N. and Tague, B. A. (1978). UNIX time-sharing system: Forward. *Bell System Technical Journal*, 57(6), pp. 1899–1904.

- [21] Spinellis, D. (2017). State of the art: a taxonomy of software tools. *IEEE Software*, 34(5), pp. 10–11.
- [22] Kochhar, P. S., Thung, F. and Lo, D. (2016). Code coverage and test suite effectiveness: empirical study with real bugs in large systems. In *Proceedings of the 22nd International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 560–564. IEEE.
- [23] Hilton, M., Tunnell, T., Huang, K., Marinov, D. and Dig, D. (2016). Usage, costs, and benefits of continuous integration in open-source projects. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 426–437. IEEE.
- [24] Beller, M., Gousios, G. and Zaidman, A. (2019). Travistorrent: Synthesizing Travis CI and GitHub for full-stack research on continuous integration. In *Proceedings of the 14th International Conference on Mining Software Repositories (MSR)*, pp. 447–450. IEEE.
- [25] Zhao, Y., Serebrenik, A., Zhou, Y., Filkov, V. and Vasilescu, B. (2017). The impact of continuous integration on other software development practices: a large-scale empirical study. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 60–71. IEEE.
- [26] Hoare, C. A. R. (1978). Communicating sequential processes. *Communications of the ACM*, 21(8), pp. 666–677.
- [27] Milner, R. (1989). *Communication and Concurrency*. Prentice Hall.
- [28] Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), pp. 199–220.
- [29] Noy, N. F. and McGuinness, D. L. (2001). Ontology development 101: a guide to creating your first ontology. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.
- [30] Dekel, U. and Herbsleb, J. D. (2009). Improving API documentation usability with knowledge pushing. In *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pp. 320–330. IEEE.